```
from dataclasses import dataclass
from typing import Iterator, Tuple
import torch
import torch.nn as nn
import torch.optim as optim
def _calculate_reach_gae(
    gamma: float,
    lam: float,
    g_seq: torch.Tensor,
    value_seq: torch.Tensor,
    done_seq: torch.Tensor,
    h_seq: torch.Tensor,
) -> Tuple[torch.Tensor, torch.Tensor]:
    """
    Reach-Avoid 任务的广义优势估计（GAE）计算。
    这是标准 GAE 的扩展版本，同时考虑到达目标（g-values）和避障安全（h-values）的双重目标。
    算法基于 JAX 参考实现 calculate_gae_reach4 移植到 PyTorch。
    数学原理：
    - 对于每个时间步 t，计算值函数目标 Q_t = max(h_t, min(g_t, γ * V_{t+1}))
    - 物理意义：值函数应反映"安全约束"和"到达目标进度"中的较大者
    - 优化目标：在保证安全（h_t < 0）的前提下，最小化到达代价（g_t）
    - 其中 g_t 是到达目标的代价，h_t 是避障安全约束
    - 使用 GAE 系数平滑多步回报，平衡偏差和方差.
    输入参数：
        gamma: float - 折扣因子，控制未来回报的重要性
        lam: float - GAE 参数，控制优势估计的偏差-方差权衡（λ∈[0,1]）
        g_seq: torch.Tensor - 形状 (horizon+1, num_envs)，到达目标函数值序列
        value_seq: torch.Tensor - 形状 (horizon+1, num_envs)，价值函数预测序列
        done_seq: torch.Tensor - 形状 (horizon, num_envs)，终止标志（环境终止或安全违规）
        h_seq: torch.Tensor - 形状 (horizon+1, num_envs)，避障安全函数值序列
    返回：
        Tuple[torch.Tensor, torch.Tensor] - (advantages, q_targets)
            advantages: 形状 (horizon, num_envs)，优势估计值
            q_targets: 形状 (horizon, num_envs)，Q 值目标用于值函数训练
    注意：
        - g_seq 和 h_seq 比 value_seq 多一个时间步（包含终止状态）
        - 当 h_t >= 0 时表示安全约束被违反，应终止回合
        - 算法从后向前计算，处理终止状态对回报传播的影响
    """
    device = value_seq.device
    dtype = value_seq.dtype
    horizon = g_seq.shape[0] - 1
    num_envs = g_seq.shape[1]
    if horizon <= 0:
        empty = torch.zeros(0, num_envs, device=device, dtype=dtype)
        return empty, empty
    lam_ratio = lam / max(1.0 - lam, 1e-6)
    gamma_tensor = value_seq.new_tensor(gamma)
    gae_coeffs = torch.zeros(horizon + 1, num_envs, device=device, dtype=dtype)
    value_table = torch.zeros_like(gae_coeffs)
```

```
    value_table[0] = value_seq[-1]
    prev_done = torch.zeros(num_envs, device=device, dtype=dtype)
    q_targets = torch.zeros(horizon, num_envs, device=device, dtype=dtype)
    index_mask = torch.arange(horizon + 1, device=device).unsqueeze(1)
    for idx in range(horizon - 1, -1, -1):
        done_row = done_seq[idx].to(dtype)
        done_row_unsqueezed = done_row.unsqueeze(0)
        prev_done_unsqueezed = prev_done.unsqueeze(0)
        rolled = torch.roll(gae_coeffs, shifts=1, dims=0)
        gae_coeffs = (
            rolled * lam * (1.0 - prev_done_unsqueezed)
            + rolled * lam_ratio * prev_done_unsqueezed
        ) * (1.0 - done_row_unsqueezed)
        gae_coeffs[0] = 1.0
        mask = (index_mask < (idx + 1)).to(dtype)
        disc_to_gh = gamma_tensor * value_table
        vhs_row = torch.maximum(
            h_seq[idx].unsqueeze(0),
            torch.minimum(g_seq[idx].unsqueeze(0), disc_to_gh),
        )
        vhs_row = vhs_row * mask
        coeff_sum = gae_coeffs.sum(dim=0, keepdim=True).clamp_min(1e-8)
        norm_coeffs = gae_coeffs / coeff_sum
        q_targets[idx] = (vhs_row * norm_coeffs).sum(dim=0)
        vhs_row = torch.roll(vhs_row, shifts=1, dims=0)
        vhs_row[0] = value_seq[idx + 1]
        value_table = vhs_row
        prev_done = done_row
    advantages = q_targets - value_seq[:-1]
    return advantages, q_targets
@dataclass
class ReachAvoidBatch:
    observations: torch.Tensor
    actions: torch.Tensor
    old_log_probs: torch.Tensor
    old_values: torch.Tensor
    advantages: torch.Tensor
    returns: torch.Tensor
class ReachAvoidBuffer:
    def __init__(self, num_envs: int, horizon: int, obs_shape: Tuple[int, ...], action_shape:
    Tuple[int, ...], device: torch.device):
        self.num_envs = num_envs
        self.horizon = horizon
        self.device = device
        obs_dim = obs_shape[0]
        act_dim = action_shape[0]
        self.observations = torch.zeros(horizon + 1, num_envs, obs_dim, device=device)
        self.actions = torch.zeros(horizon, num_envs, act_dim, device=device)
        self.log_probs = torch.zeros(horizon, num_envs, device=device)
        self.values = torch.zeros(horizon, num_envs, device=device)
```

```
        self.advantages = torch.zeros(horizon, num_envs, device=device)
        self.returns = torch.zeros(horizon, num_envs, device=device)
        self.g_values = torch.zeros(horizon + 1, num_envs, device=device)
        self.h_values = torch.zeros(horizon + 1, num_envs, device=device)
        self.dones = torch.zeros(horizon, num_envs, device=device, dtype=torch.bool)
        self.step = 0
    def clear(self) -> None:
        self.step = 0
    def add(
        self,
        obs: torch.Tensor,
        actions: torch.Tensor,
        log_probs: torch.Tensor,
        values: torch.Tensor,
        g_values: torch.Tensor,
        h_values: torch.Tensor,
        dones: torch.Tensor,
        next_obs: torch.Tensor,
        next_g: torch.Tensor,
        next_h: torch.Tensor,
    ) -> None:
        idx = self.step
        self.observations[idx] = obs
        self.actions[idx] = actions
        self.log_probs[idx] = log_probs
        self.values[idx] = values
        self.g_values[idx] = g_values
        self.h_values[idx] = h_values
        self.dones[idx] = dones.bool()
        self.observations[idx + 1] = next_obs
        self.g_values[idx + 1] = next_g
        self.h_values[idx + 1] = next_h
        self.step += 1
    def store_rollout(
        self,
        observations: torch.Tensor,
        actions: torch.Tensor,
        log_probs: torch.Tensor,
        values: torch.Tensor,
        g_values: torch.Tensor,
        h_values: torch.Tensor,
        dones: torch.Tensor,
    ) -> None:
        """
        批量存储完整的 rollout 轨迹数据到缓冲区。
        与 add() 方法逐步添加单步数据不同，此方法一次性存入整个 horizon 长度的轨迹。
        常用于外部已经收集好完整轨迹的场景，或从预收集数据加载。
        """
        if observations.shape != self.observations.shape:
            raise ValueError("observations shape mismatch")
```

```
        if actions.shape != self.actions.shape:
            raise ValueError("actions shape mismatch")
        if log_probs.shape != self.log_probs.shape:
            raise ValueError("log_probs shape mismatch")
        if values.shape != self.values.shape:
            raise ValueError("values shape mismatch")
        if g_values.shape != self.g_values.shape:
            raise ValueError("g_values shape mismatch")
        if h_values.shape != self.h_values.shape:
            raise ValueError("h_values shape mismatch")
        if dones.shape != self.dones.shape:
            raise ValueError("dones shape mismatch")
        self.observations.copy_(observations)
        self.actions.copy_(actions)
        self.log_probs.copy_(log_probs)
        self.values.copy_(values)
        self.g_values.copy_(g_values)
        self.h_values.copy_(h_values)
        self.dones.copy_(dones.bool())
        self.step = self.horizon
    def compute_advantages(self, last_values: torch.Tensor, gamma: float, lam: float) -> None:
        if self.step != self.horizon:
            raise RuntimeError("incomplete rollout stored in buffer")
        value_seq = torch.cat((self.values, last_values.unsqueeze(0)), dim=0)
        env_dones = self.dones
        safety_dones = self.h_values[:-1] >= 0
        done_seq = torch.logical_or(env_dones, safety_dones)
        adv, targets = _calculate_reach_gae(gamma, lam, self.g_values, value_seq, done_seq,
    self.h_values)
        self.advantages.copy_(adv)
        self.returns.copy_(targets)
    def _flat_view(self) -> ReachAvoidBatch:
        obs = self.observations[:-1].reshape(-1, self.observations.size(-1))
        actions = self.actions.reshape(-1, self.actions.size(-1))
        log_probs = self.log_probs.reshape(-1)
        values = self.values.reshape(-1)
        advantages = self.advantages.reshape(-1)
        returns = self.returns.reshape(-1)
        return ReachAvoidBatch(obs, actions, log_probs, values, advantages, returns)
    def iter_batches(self, num_mini_batches: int, num_epochs: int) -> Iterator[ReachAvoidBatch]:
        """
        生成用于PPO多轮小批量训练的数据批次。
        PPO算法通常进行多轮（epochs）训练，每轮中将所有数据随机打乱后分割成多个小批量（mini-batches）。
        这种设计提高数据利用率，减少过拟合，并通过随机化改善训练稳定性。
        参数:
            num_mini_batches: 每个训练轮次中的小批量数量
            num_epochs: 训练轮次数量（对同一批数据进行多轮训练）
        返回:
            生成器，每次迭代返回一个ReachAvoidBatch数据对象
        """
```

```
        data = self._flat_view()
        batch_size = data.observations.size(0)
        mini_batch_size = batch_size // num_mini_batches
        for _ in range(num_epochs):
            indices = torch.randperm(batch_size, device=self.device)
            for start in range(0, batch_size, mini_batch_size):
                end = start + mini_batch_size
                idx = indices[start:end]
                yield ReachAvoidBatch(
                    data.observations[idx],
                    data.actions[idx],
                    data.old_log_probs[idx],
                    data.old_values[idx],
                    data.advantages[idx],
                    data.returns[idx],
                )
class ReachAvoidPPO:
    def __init__(
        self,
        actor_critic,
        learning_rate: float = 3e-4,
        gamma: float = 0.999,
        lam: float = 0.95,
        num_learning_epochs: int = 4,
        num_mini_batches: int = 4,
        clip_param: float = 0.2,
        value_loss_coef: float = 1.0,
        entropy_coef: float = 0.0,
        max_grad_norm: float = 1.0,
        device: str = "cpu",
        **kwargs,
    ) -> None:
        self.device = torch.device(device)
        self.actor_critic = actor_critic.to(self.device)
        self.learning_rate = learning_rate
        self.gamma = gamma
        self.lam = lam
        self.num_learning_epochs = num_learning_epochs
        self.num_mini_batches = num_mini_batches
        self.clip_param = clip_param
        self.value_loss_coef = value_loss_coef
        self.entropy_coef = entropy_coef
        self.max_grad_norm = max_grad_norm
        self.optimizer = optim.Adam(self.actor_critic.parameters(), lr=self.learning_rate)
        self.buffer = None
        self.last_value_stats = {}
    def init_storage(self, num_envs: int, horizon: int, obs_shape, action_shape) -> None:
        obs_shape = tuple(obs_shape) if isinstance(obs_shape, (list, tuple)) else (obs_shape,)
        action_shape = tuple(action_shape) if isinstance(action_shape, (list, tuple)) else
    (action_shape,)
```

```
        self.buffer = ReachAvoidBuffer(num_envs, horizon, obs_shape, action_shape, self.device)
    def act(self, obs: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        with torch.no_grad():
            actions = self.actor_critic.act(obs)
            log_probs = self.actor_critic.get_actions_log_prob(actions)
            values = self.actor_critic.evaluate(obs).squeeze(-1)
        return actions, log_probs, values
    def update(self) -> Tuple[float, float]:
        """
        PPO 算法核心更新步骤（针对 Reach-Avoid 任务的变体）。
        该函数不负责数据采集（由外部环境和 buffer 完成），只在完整 rollout 可用时被调用以执行参数更新。
        """
        assert self.buffer is not None
        advantages = self.buffer.advantages
        with torch.no_grad():
            values_flat = self.buffer.values.reshape(-1)
            returns_flat = self.buffer.returns.reshape(-1)
            diff = returns_flat - values_flat
            value_mean = values_flat.mean()
            value_std = values_flat.std(unbiased=False)
            return_mean = returns_flat.mean()
            return_std = returns_flat.std(unbiased=False)
            value_rmse = diff.pow(2).mean().sqrt()
            var_returns = returns_flat.var(unbiased=False)
            diff_var = diff.var(unbiased=False)
            if var_returns.item() > 1e-8:
                explained_variance = 1.0 - diff_var / var_returns
            else:
                explained_variance = torch.tensor(0.0, device=values_flat.device)
            adv_mean = advantages.mean()
            adv_std = advantages.std(unbiased=False)
            self.last_value_stats = {
                "value_mean": value_mean.item(),
                "value_std": value_std.item(),
                "return_mean": return_mean.item(),
                "return_std": return_std.item(),
                "value_rmse": value_rmse.item(),
                "explained_variance": explained_variance.item(),
                "adv_mean": adv_mean.item(),
                "adv_std": adv_std.item(),
            }
        advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)
        self.buffer.advantages.copy_(advantages)
        policy_loss_acc = 0.0
        value_loss_acc = 0.0
        batch_count = 0
        for batch in self.buffer.iter_batches(self.num_mini_batches, self.num_learning_epochs):
            obs_batch = batch.observations
            act_batch = batch.actions
            old_log_probs = batch.old_log_probs
```

```
            returns_batch = batch.returns
            old_values = batch.old_values
            adv_batch = batch.advantages
            self.actor_critic.update_distribution(obs_batch)
            log_probs = self.actor_critic.get_actions_log_prob(act_batch)
            entropy = self.actor_critic.entropy.mean()
            values = self.actor_critic.evaluate(obs_batch).squeeze(-1)
            gae_batch = -adv_batch
            ratio = torch.exp(log_probs - old_log_probs)
            loss_actor1 = ratio * gae_batch
            loss_actor2 = torch.clamp(ratio, 1.0 - self.clip_param, 1.0 + self.clip_param) *
    gae_batch
            policy_loss = -torch.min(loss_actor1, loss_actor2).mean()
            values_clipped = old_values + torch.clamp(values - old_values, -self.clip_param,
    self.clip_param)
            value_loss_unclipped = (values - returns_batch).pow(2)
            value_loss_clipped = (values_clipped - returns_batch).pow(2)
            value_loss = 0.5 * torch.max(value_loss_unclipped, value_loss_clipped).mean()
            loss = policy_loss + self.value_loss_coef * value_loss - self.entropy_coef * entropy
            self.optimizer.zero_grad()
            loss.backward()
            nn.utils.clip_grad_norm_(self.actor_critic.parameters(), self.max_grad_norm)
            self.optimizer.step()
            policy_loss_acc += policy_loss.item()
            value_loss_acc += value_loss.item()
            batch_count += 1
        mean_policy_loss = policy_loss_acc / max(batch_count, 1)
        mean_value_loss = value_loss_acc / max(batch_count, 1)
        self.buffer.clear()
        return mean_policy_loss, mean_value_loss
import torch
import torch.nn as nn
import torch.optim as optim
from rsl_rl.modules import ActorCritic
from rsl_rl.storage import RolloutStorage
class PPO:
    actor_critic: ActorCritic
    def __init__(self,
                 actor_critic,
                 num_learning_epochs=1,
                 num_mini_batches=1,
                 clip_param=0.2,
                 gamma=0.998,
                 lam=0.95,
                 value_loss_coef=1.0,
                 entropy_coef=0.0,
                 learning_rate=1e-3,
                 max_grad_norm=1.0,
                 use_clipped_value_loss=True,
                 schedule="fixed",
```

```
                desired_kl=0.01,
                device='cpu',
                ):
        self.device = device
        self.desired_kl = desired_kl
        self.schedule = schedule
        self.learning_rate = learning_rate
        self.actor_critic = actor_critic
        self.actor_critic.to(self.device)
        self.storage = None
        self.optimizer = optim.Adam(self.actor_critic.parameters(), lr=learning_rate)
        self.transition = RolloutStorage.Transition()
        self.clip_param = clip_param
        self.num_learning_epochs = num_learning_epochs
        self.num_mini_batches = num_mini_batches
        self.value_loss_coef = value_loss_coef
        self.entropy_coef = entropy_coef
        self.gamma = gamma
        self.lam = lam
        self.max_grad_norm = max_grad_norm
        self.use_clipped_value_loss = use_clipped_value_loss
    def init_storage(self, num_envs, num_transitions_per_env, actor_obs_shape, critic_obs_shape,
    action_shape):
        self.storage = RolloutStorage(num_envs, num_transitions_per_env, actor_obs_shape,
    critic_obs_shape, action_shape, self.device)
    def test_mode(self):
        self.actor_critic.test()
    def train_mode(self):
        self.actor_critic.train()
    def act(self, obs, critic_obs):
        if self.actor_critic.is_recurrent:
            self.transition.hidden_states = self.actor_critic.get_hidden_states()
        self.transition.actions = self.actor_critic.act(obs).detach()
        self.transition.values = self.actor_critic.evaluate(critic_obs).detach()
        self.transition.actions_log_prob =
    self.actor_critic.get_actions_log_prob(self.transition.actions).detach()
        self.transition.action_mean = self.actor_critic.action_mean.detach()
        self.transition.action_sigma = self.actor_critic.action_std.detach()
        self.transition.observations = obs
        self.transition.critic_observations = critic_obs
        return self.transition.actions
    def process_env_step(self, rewards, dones, infos):
        self.transition.rewards = rewards.clone()
        self.transition.dones = dones
        if 'time_outs' in infos:
            self.transition.rewards += self.gamma * torch.squeeze(self.transition.values *
    infos['time_outs'].unsqueeze(1).to(self.device), 1)
        self.storage.add_transitions(self.transition)
        self.transition.clear()
        self.actor_critic.reset(dones)
```

```
    def compute_returns(self, last_critic_obs):
        last_values= self.actor_critic.evaluate(last_critic_obs).detach()
        self.storage.compute_returns(last_values, self.gamma, self.lam)
    def update(self):
        mean_value_loss = 0
        mean_surrogate_loss = 0
        if self.actor_critic.is_recurrent:
            generator = self.storage.reccurent_mini_batch_generator(self.num_mini_batches,
self.num_learning_epochs)
        else:
            generator = self.storage.mini_batch_generator(self.num_mini_batches,
self.num_learning_epochs)
        for obs_batch, critic_obs_batch, actions_batch, target_values_batch, advantages_batch,
returns_batch, old_actions_log_prob_batch, \
            old_mu_batch, old_sigma_batch, hid_states_batch, masks_batch in generator:
                self.actor_critic.act(obs_batch, masks=masks_batch,
hidden_states=hid_states_batch[0])
                actions_log_prob_batch = self.actor_critic.get_actions_log_prob(actions_batch)
                value_batch = self.actor_critic.evaluate(critic_obs_batch, masks=masks_batch,
hidden_states=hid_states_batch[1])
                mu_batch = self.actor_critic.action_mean
                sigma_batch = self.actor_critic.action_std
                entropy_batch = self.actor_critic.entropy
                if self.desired_kl != None and self.schedule == 'adaptive':
                    with torch.inference_mode():
                        kl = torch.sum(
                            torch.log(sigma_batch / old_sigma_batch + 1.e-5) +
(torch.square(old_sigma_batch) + torch.square(old_mu_batch - mu_batch)) / (2.0 *
torch.square(sigma_batch)) - 0.5, axis=-1)
                        kl_mean = torch.mean(kl)
                        if kl_mean > self.desired_kl * 2.0:
                            self.learning_rate = max(1e-5, self.learning_rate / 1.5)
                        elif kl_mean < self.desired_kl / 2.0 and kl_mean > 0.0:
                            self.learning_rate = min(1e-2, self.learning_rate * 1.5)
                        for param_group in self.optimizer.param_groups:
                            param_group['lr'] = self.learning_rate
                ratio = torch.exp(actions_log_prob_batch -
torch.squeeze(old_actions_log_prob_batch))
                surrogate = -torch.squeeze(advantages_batch) * ratio
                surrogate_clipped = -torch.squeeze(advantages_batch) * torch.clamp(ratio, 1.0 -
self.clip_param,
                                                                                   1.0 +
self.clip_param)
                surrogate_loss = torch.max(surrogate, surrogate_clipped).mean()
                if self.use_clipped_value_loss:
                    value_clipped = target_values_batch + (value_batch -
target_values_batch).clamp(-self.clip_param,

self.clip_param)
                    value_losses = (value_batch - returns_batch).pow(2)
```

```
                    value_losses_clipped = (value_clipped - returns_batch).pow(2)
                    value_loss = torch.max(value_losses, value_losses_clipped).mean()
                else:
                    value_loss = (returns_batch - value_batch).pow(2).mean()
                loss = surrogate_loss + self.value_loss_coef * value_loss - self.entropy_coef *
    entropy_batch.mean()
                self.optimizer.zero_grad()
                loss.backward()
                nn.utils.clip_grad_norm_(self.actor_critic.parameters(), self.max_grad_norm)
                self.optimizer.step()
                mean_value_loss += value_loss.item()
                mean_surrogate_loss += surrogate_loss.item()
        num_updates = self.num_learning_epochs * self.num_mini_batches
        mean_value_loss /= num_updates
        mean_surrogate_loss /= num_updates
        self.storage.clear()
        return mean_value_loss, mean_surrogate_loss
import numpy as np
import torch
import torch.nn as nn
from torch.distributions import Normal
from torch.nn.modules import rnn
class ActorCritic(nn.Module):
    is_recurrent = False
    def __init__(self,  num_actor_obs,
                        num_critic_obs,
                        num_actions,
                        actor_hidden_dims=[256, 256, 256],
                        critic_hidden_dims=[256, 256, 256],
                        activation='elu',
                        init_noise_std=1.0,
                        **kwargs):
        if kwargs:
            print("ActorCritic.__init__ got unexpected arguments, which will be ignored: " +
    str([key for key in kwargs.keys()]))
        super(ActorCritic, self).__init__()
        activation = get_activation(activation)
        mlp_input_dim_a = num_actor_obs
        mlp_input_dim_c = num_critic_obs
        actor_layers = []
        actor_layers.append(nn.Linear(mlp_input_dim_a, actor_hidden_dims[0]))
        actor_layers.append(activation)
        for l in range(len(actor_hidden_dims)):
            if l == len(actor_hidden_dims) - 1:
                actor_layers.append(nn.Linear(actor_hidden_dims[l], num_actions))
            else:
                actor_layers.append(nn.Linear(actor_hidden_dims[l], actor_hidden_dims[l + 1]))
                actor_layers.append(activation)
        self.actor = nn.Sequential(*actor_layers)
        critic_layers = []
```

```
        critic_layers.append(nn.Linear(mlp_input_dim_c, critic_hidden_dims[0]))
        critic_layers.append(activation)
        for l in range(len(critic_hidden_dims)):
            if l == len(critic_hidden_dims) - 1:
                critic_layers.append(nn.Linear(critic_hidden_dims[l], 1))
            else:
                critic_layers.append(nn.Linear(critic_hidden_dims[l], critic_hidden_dims[l + 1]))
                critic_layers.append(activation)
        self.critic = nn.Sequential(*critic_layers)
        print(f"Actor MLP: {self.actor}")
        print(f"Critic MLP: {self.critic}")
        self.std = nn.Parameter(init_noise_std * torch.ones(num_actions))
        self.distribution = None
        Normal.set_default_validate_args = False
    @staticmethod
    def init_weights(sequential, scales):
        [torch.nn.init.orthogonal_(module.weight, gain=scales[idx]) for idx, module in
         enumerate(mod for mod in sequential if isinstance(mod, nn.Linear))]
    def reset(self, dones=None):
        pass
    def forward(self):
        raise NotImplementedError
    @property
    def action_mean(self):
        return self.distribution.mean
    @property
    def action_std(self):
        return self.distribution.stddev
    @property
    def entropy(self):
        return self.distribution.entropy().sum(dim=-1)
    def update_distribution(self, observations):
        mean = self.actor(observations)
        self.distribution = Normal(mean, mean*0. + self.std)
    def act(self, observations, **kwargs):
        self.update_distribution(observations)
        return self.distribution.sample()
    def get_actions_log_prob(self, actions):
        return self.distribution.log_prob(actions).sum(dim=-1)
    def act_inference(self, observations):
        actions_mean = self.actor(observations)
        return actions_mean
    def evaluate(self, critic_observations, **kwargs):
        value = self.critic(critic_observations)
        return value
def get_activation(act_name):
    if act_name == "elu":
        return nn.ELU()
    elif act_name == "selu":
        return nn.SELU()
```

```
    elif act_name == "relu":
        return nn.ReLU()
    elif act_name == "crelu":
        return nn.ReLU()
    elif act_name == "lrelu":
        return nn.LeakyReLU()
    elif act_name == "tanh":
        return nn.Tanh()
    elif act_name == "sigmoid":
        return nn.Sigmoid()
    else:
        print("invalid activation function!")
        return None
import numpy as np
import torch
import torch.nn as nn
from torch.distributions import Normal
from torch.nn.modules import rnn
from .actor_critic import ActorCritic, get_activation
from rsl_rl.utils import unpad_trajectories
class ActorCriticRecurrent(ActorCritic):
    is_recurrent = True
    def __init__(self,  num_actor_obs,
                        num_critic_obs,
                        num_actions,
                        actor_hidden_dims=[256, 256, 256],
                        critic_hidden_dims=[256, 256, 256],
                        activation='elu',
                        rnn_type='lstm',
                        rnn_hidden_size=256,
                        rnn_num_layers=1,
                        init_noise_std=1.0,
                        **kwargs):
        if kwargs:
            print("ActorCriticRecurrent.__init__ got unexpected arguments, which will be ignored: "
    + str(kwargs.keys()),)
        super().__init__(num_actor_obs=rnn_hidden_size,
                         num_critic_obs=rnn_hidden_size,
                         num_actions=num_actions,
                         actor_hidden_dims=actor_hidden_dims,
                         critic_hidden_dims=critic_hidden_dims,
                         activation=activation,
                         init_noise_std=init_noise_std)
        activation = get_activation(activation)
        self.memory_a = Memory(num_actor_obs, type=rnn_type, num_layers=rnn_num_layers,
    hidden_size=rnn_hidden_size)
        self.memory_c = Memory(num_critic_obs, type=rnn_type, num_layers=rnn_num_layers,
    hidden_size=rnn_hidden_size)
        print(f"Actor RNN: {self.memory_a}")
        print(f"Critic RNN: {self.memory_c}")
```

```
    def reset(self, dones=None):
        self.memory_a.reset(dones)
        self.memory_c.reset(dones)
    def act(self, observations, masks=None, hidden_states=None):
        input_a = self.memory_a(observations, masks, hidden_states)
        return super().act(input_a.squeeze(0))
    def act_inference(self, observations):
        input_a = self.memory_a(observations)
        return super().act_inference(input_a.squeeze(0))
    def evaluate(self, critic_observations, masks=None, hidden_states=None):
        input_c = self.memory_c(critic_observations, masks, hidden_states)
        return super().evaluate(input_c.squeeze(0))
    def get_hidden_states(self):
        return self.memory_a.hidden_states, self.memory_c.hidden_states
class Memory(torch.nn.Module):
    def __init__(self, input_size, type='lstm', num_layers=1, hidden_size=256):
        super().__init__()
        rnn_cls = nn.GRU if type.lower() == 'gru' else nn.LSTM
        self.rnn = rnn_cls(input_size=input_size, hidden_size=hidden_size, num_layers=num_layers)
        self.hidden_states = None
    def forward(self, input, masks=None, hidden_states=None):
        batch_mode = masks is not None
        if batch_mode:
            if hidden_states is None:
                raise ValueError("Hidden states not passed to memory module during policy update")
            out, _ = self.rnn(input, hidden_states)
            out = unpad_trajectories(out, masks)
        else:
            out, self.hidden_states = self.rnn(input.unsqueeze(0), self.hidden_states)
        return out
    def reset(self, dones=None):
        for hidden_state in self.hidden_states:
            hidden_state[..., dones, :] = 0.0
import torch
import numpy as np
from rsl_rl.utils import split_and_pad_trajectories
class RolloutStorage:
    class Transition:
        def __init__(self):
            self.observations = None
            self.critic_observations = None
            self.actions = None
            self.rewards = None
            self.dones = None
            self.values = None
            self.actions_log_prob = None
            self.action_mean = None
            self.action_sigma = None
            self.hidden_states = None
        def clear(self):
```

```
            self.__init__()
    def __init__(self, num_envs, num_transitions_per_env, obs_shape, privileged_obs_shape,
    actions_shape, device='cpu'):
        self.device = device
        self.obs_shape = obs_shape
        self.privileged_obs_shape = privileged_obs_shape
        self.actions_shape = actions_shape
        self.observations = torch.zeros(num_transitions_per_env, num_envs, *obs_shape,
    device=self.device)
        if privileged_obs_shape[0] is not None:
            self.privileged_observations = torch.zeros(num_transitions_per_env, num_envs,
    *privileged_obs_shape, device=self.device)
        else:
            self.privileged_observations = None
        self.rewards = torch.zeros(num_transitions_per_env, num_envs, 1, device=self.device)
        self.actions = torch.zeros(num_transitions_per_env, num_envs, *actions_shape,
    device=self.device)
        self.dones = torch.zeros(num_transitions_per_env, num_envs, 1, device=self.device).byte()
        self.actions_log_prob = torch.zeros(num_transitions_per_env, num_envs, 1,
    device=self.device)
        self.values = torch.zeros(num_transitions_per_env, num_envs, 1, device=self.device)
        self.returns = torch.zeros(num_transitions_per_env, num_envs, 1, device=self.device)
        self.advantages = torch.zeros(num_transitions_per_env, num_envs, 1, device=self.device)
        self.mu = torch.zeros(num_transitions_per_env, num_envs, *actions_shape, device=self.device)
        self.sigma = torch.zeros(num_transitions_per_env, num_envs, *actions_shape,
    device=self.device)
        self.num_transitions_per_env = num_transitions_per_env
        self.num_envs = num_envs
        self.saved_hidden_states_a = None
        self.saved_hidden_states_c = None
        self.step = 0
    def add_transitions(self, transition: Transition):
        if self.step >= self.num_transitions_per_env:
            raise AssertionError("Rollout buffer overflow")
        self.observations[self.step].copy_(transition.observations)
        if self.privileged_observations is not None:
    self.privileged_observations[self.step].copy_(transition.critic_observations)
        self.actions[self.step].copy_(transition.actions)
        self.rewards[self.step].copy_(transition.rewards.view(-1, 1))
        self.dones[self.step].copy_(transition.dones.view(-1, 1))
        self.values[self.step].copy_(transition.values)
        self.actions_log_prob[self.step].copy_(transition.actions_log_prob.view(-1, 1))
        self.mu[self.step].copy_(transition.action_mean)
        self.sigma[self.step].copy_(transition.action_sigma)
        self._save_hidden_states(transition.hidden_states)
        self.step += 1
    def _save_hidden_states(self, hidden_states):
        if hidden_states is None or hidden_states==(None, None):
            return
        hid_a = hidden_states[0] if isinstance(hidden_states[0], tuple) else (hidden_states[0],)
```

```
        hid_c = hidden_states[1] if isinstance(hidden_states[1], tuple) else (hidden_states[1],)
        if self.saved_hidden_states_a is None:
            self.saved_hidden_states_a = [torch.zeros(self.observations.shape[0], *hid_a[i].shape,
    device=self.device) for i in range(len(hid_a))]
            self.saved_hidden_states_c = [torch.zeros(self.observations.shape[0], *hid_c[i].shape,
    device=self.device) for i in range(len(hid_c))]
        for i in range(len(hid_a)):
            self.saved_hidden_states_a[i][self.step].copy_(hid_a[i])
            self.saved_hidden_states_c[i][self.step].copy_(hid_c[i])
    def clear(self):
        self.step = 0
    def compute_returns(self, last_values, gamma, lam):
        advantage = 0
        for step in reversed(range(self.num_transitions_per_env)):
            if step == self.num_transitions_per_env - 1:
                next_values = last_values
            else:
                next_values = self.values[step + 1]
            next_is_not_terminal = 1.0 - self.dones[step].float()
            delta = self.rewards[step] + next_is_not_terminal * gamma * next_values -
    self.values[step]
            advantage = delta + next_is_not_terminal * gamma * lam * advantage
            self.returns[step] = advantage + self.values[step]
        self.advantages = self.returns - self.values
        self.advantages = (self.advantages - self.advantages.mean()) / (self.advantages.std() +
    1e-8)
    def get_statistics(self):
        done = self.dones
        done[-1] = 1
        flat_dones = done.permute(1, 0, 2).reshape(-1, 1)
        done_indices = torch.cat((flat_dones.new_tensor([-1], dtype=torch.int64),
    flat_dones.nonzero(as_tuple=False)[:, 0]))
        trajectory_lengths = (done_indices[1:] - done_indices[:-1])
        return trajectory_lengths.float().mean(), self.rewards.mean()
    def mini_batch_generator(self, num_mini_batches, num_epochs=8):
        batch_size = self.num_envs * self.num_transitions_per_env
        mini_batch_size = batch_size // num_mini_batches
        indices = torch.randperm(num_mini_batches*mini_batch_size, requires_grad=False,
    device=self.device)
        observations = self.observations.flatten(0, 1)
        if self.privileged_observations is not None:
            critic_observations = self.privileged_observations.flatten(0, 1)
        else:
            critic_observations = observations
        actions = self.actions.flatten(0, 1)
        values = self.values.flatten(0, 1)
        returns = self.returns.flatten(0, 1)
        old_actions_log_prob = self.actions_log_prob.flatten(0, 1)
        advantages = self.advantages.flatten(0, 1)
        old_mu = self.mu.flatten(0, 1)
```

```
        old_sigma = self.sigma.flatten(0, 1)
        for epoch in range(num_epochs):
            for i in range(num_mini_batches):
                start = i*mini_batch_size
                end = (i+1)*mini_batch_size
                batch_idx = indices[start:end]
                obs_batch = observations[batch_idx]
                critic_observations_batch = critic_observations[batch_idx]
                actions_batch = actions[batch_idx]
                target_values_batch = values[batch_idx]
                returns_batch = returns[batch_idx]
                old_actions_log_prob_batch = old_actions_log_prob[batch_idx]
                advantages_batch = advantages[batch_idx]
                old_mu_batch = old_mu[batch_idx]
                old_sigma_batch = old_sigma[batch_idx]
                yield obs_batch, critic_observations_batch, actions_batch, target_values_batch,
    advantages_batch, returns_batch, \
                      old_actions_log_prob_batch, old_mu_batch, old_sigma_batch, (None, None), None
    def reccurent_mini_batch_generator(self, num_mini_batches, num_epochs=8):
        padded_obs_trajectories, trajectory_masks = split_and_pad_trajectories(self.observations,
    self.dones)
        if self.privileged_observations is not None:
            padded_critic_obs_trajectories, _ =
    split_and_pad_trajectories(self.privileged_observations, self.dones)
        else:
            padded_critic_obs_trajectories = padded_obs_trajectories
        mini_batch_size = self.num_envs // num_mini_batches
        for ep in range(num_epochs):
            first_traj = 0
            for i in range(num_mini_batches):
                start = i*mini_batch_size
                stop = (i+1)*mini_batch_size
                dones = self.dones.squeeze(-1)
                last_was_done = torch.zeros_like(dones, dtype=torch.bool)
                last_was_done[1:] = dones[:-1]
                last_was_done[0] = True
                trajectories_batch_size = torch.sum(last_was_done[:, start:stop])
                last_traj = first_traj + trajectories_batch_size
                masks_batch = trajectory_masks[:, first_traj:last_traj]
                obs_batch = padded_obs_trajectories[:, first_traj:last_traj]
                critic_obs_batch = padded_critic_obs_trajectories[:, first_traj:last_traj]
                actions_batch = self.actions[:, start:stop]
                old_mu_batch = self.mu[:, start:stop]
                old_sigma_batch = self.sigma[:, start:stop]
                returns_batch = self.returns[:, start:stop]
                advantages_batch = self.advantages[:, start:stop]
                values_batch = self.values[:, start:stop]
                old_actions_log_prob_batch = self.actions_log_prob[:, start:stop]
                last_was_done = last_was_done.permute(1, 0)
                hid_a_batch = [ saved_hidden_states.permute(2, 0, 1,
```

```
    3)[last_was_done][first_traj:last_traj].transpose(1, 0).contiguous()
                                for saved_hidden_states in self.saved_hidden_states_a ]
                hid_c_batch = [ saved_hidden_states.permute(2, 0, 1,
    3)[last_was_done][first_traj:last_traj].transpose(1, 0).contiguous()
                                for saved_hidden_states in self.saved_hidden_states_c ]
                hid_a_batch = hid_a_batch[0] if len(hid_a_batch)==1 else hid_a_batch
                hid_c_batch = hid_c_batch[0] if len(hid_c_batch)==1 else hid_a_batch
                yield obs_batch, critic_obs_batch, actions_batch, values_batch, advantages_batch,
    returns_batch, \
                        old_actions_log_prob_batch, old_mu_batch, old_sigma_batch, (hid_a_batch,
    hid_c_batch), masks_batch
                first_traj = last_traj
import time
import os
from collections import deque
import statistics
from torch.utils.tensorboard import SummaryWriter
import torch
from rsl_rl.algorithms import PPO
from rsl_rl.modules import ActorCritic, ActorCriticRecurrent
from rsl_rl.env import VecEnv
class OnPolicyRunner:
    def __init__(self,
                 env: VecEnv,
                 train_cfg,
                 log_dir=None,
                 device='cpu'):
        self.cfg=train_cfg["runner"]
        self.alg_cfg = train_cfg["algorithm"]
        self.policy_cfg = train_cfg["policy"]
        self.device = device
        self.env = env
        if self.env.num_privileged_obs is not None:
            num_critic_obs = self.env.num_privileged_obs
        else:
            num_critic_obs = self.env.num_obs
        actor_critic_class = eval(self.cfg["policy_class_name"])
        actor_critic: ActorCritic = actor_critic_class( self.env.num_obs,
                                                        num_critic_obs,
                                                        self.env.num_actions,
                                                        **self.policy_cfg).to(self.device)
        alg_class = eval(self.cfg["algorithm_class_name"])
        self.alg: PPO = alg_class(actor_critic, device=self.device, **self.alg_cfg)
        self.num_steps_per_env = self.cfg["num_steps_per_env"]
        self.save_interval = self.cfg["save_interval"]
        self.alg.init_storage(self.env.num_envs, self.num_steps_per_env, [self.env.num_obs],
    [self.env.num_privileged_obs], [self.env.num_actions])
        self.log_dir = log_dir
        self.writer = None
        self.tot_timesteps = 0
```

```
        self.tot_time = 0
        self.current_learning_iteration = 0
        _, _ = self.env.reset()
    def learn(self, num_learning_iterations, init_at_random_ep_len=False):
        if self.log_dir is not None and self.writer is None:
            self.writer = SummaryWriter(log_dir=self.log_dir, flush_secs=10)
        if init_at_random_ep_len:
            self.env.episode_length_buf = torch.randint_like(self.env.episode_length_buf,
    high=int(self.env.max_episode_length))
        obs = self.env.get_observations()
        privileged_obs = self.env.get_privileged_observations()
        critic_obs = privileged_obs if privileged_obs is not None else obs
        obs, critic_obs = obs.to(self.device), critic_obs.to(self.device)
        self.alg.actor_critic.train()
        ep_infos = []
        rewbuffer = deque(maxlen=100)
        lenbuffer = deque(maxlen=100)
        cur_reward_sum = torch.zeros(self.env.num_envs, dtype=torch.float, device=self.device)
        cur_episode_length = torch.zeros(self.env.num_envs, dtype=torch.float, device=self.device)
        tot_iter = self.current_learning_iteration + num_learning_iterations
        for it in range(self.current_learning_iteration, tot_iter):
            start = time.time()
            with torch.inference_mode():
                for i in range(self.num_steps_per_env):
                    actions = self.alg.act(obs, critic_obs)
                    obs, privileged_obs, rewards, dones, infos = self.env.step(actions)
                    critic_obs = privileged_obs if privileged_obs is not None else obs
                    obs, critic_obs, rewards, dones = obs.to(self.device),
    critic_obs.to(self.device), rewards.to(self.device), dones.to(self.device)
                    self.alg.process_env_step(rewards, dones, infos)
                    if self.log_dir is not None:
                        if 'episode' in infos:
                            ep_infos.append(infos['episode'])
                        cur_reward_sum += rewards
                        cur_episode_length += 1
                        new_ids = (dones > 0).nonzero(as_tuple=False)
                        rewbuffer.extend(cur_reward_sum[new_ids][:, 0].cpu().numpy().tolist())
                        lenbuffer.extend(cur_episode_length[new_ids][:, 0].cpu().numpy().tolist())
                        cur_reward_sum[new_ids] = 0
                        cur_episode_length[new_ids] = 0
                stop = time.time()
                collection_time = stop - start
                start = stop
                self.alg.compute_returns(critic_obs)
            mean_value_loss, mean_surrogate_loss = self.alg.update()
            stop = time.time()
            learn_time = stop - start
            if self.log_dir is not None:
                self.log(locals())
            if it % self.save_interval == 0:
```

```
                self.save(os.path.join(self.log_dir, 'model_{}.pt'.format(it)))
            ep_infos.clear()
        self.current_learning_iteration += num_learning_iterations
        self.save(os.path.join(self.log_dir, 'model_{}.pt'.format(self.current_learning_iteration)))
    def log(self, locs, width=80, pad=35):
        self.tot_timesteps += self.num_steps_per_env * self.env.num_envs
        self.tot_time += locs['collection_time'] + locs['learn_time']
        iteration_time = locs['collection_time'] + locs['learn_time']
        ep_string = f''
        if locs['ep_infos']:
            for key in locs['ep_infos'][0]:
                infotensor = torch.tensor([], device=self.device)
                for ep_info in locs['ep_infos']:
                    if not isinstance(ep_info[key], torch.Tensor):
                        ep_info[key] = torch.Tensor([ep_info[key]])
                    if len(ep_info[key].shape) == 0:
                        ep_info[key] = ep_info[key].unsqueeze(0)
                    infotensor = torch.cat((infotensor, ep_info[key].to(self.device)))
                value = torch.mean(infotensor)
                self.writer.add_scalar('Episode/' + key, value, locs['it'])
                ep_string += f"""{f'Mean episode {key}:':>{pad}} {value:.4f}\n"""
        mean_std = self.alg.actor_critic.std.mean()
        fps = int(self.num_steps_per_env * self.env.num_envs / (locs['collection_time'] +
    locs['learn_time']))
        self.writer.add_scalar('Loss/value_function', locs['mean_value_loss'], locs['it'])
        self.writer.add_scalar('Loss/surrogate', locs['mean_surrogate_loss'], locs['it'])
        self.writer.add_scalar('Loss/learning_rate', self.alg.learning_rate, locs['it'])
        self.writer.add_scalar('Policy/mean_noise_std', mean_std.item(), locs['it'])
        self.writer.add_scalar('Perf/total_fps', fps, locs['it'])
        self.writer.add_scalar('Perf/collection time', locs['collection_time'], locs['it'])
        self.writer.add_scalar('Perf/learning_time', locs['learn_time'], locs['it'])
        if len(locs['rewbuffer']) > 0:
            self.writer.add_scalar('Train/mean_reward', statistics.mean(locs['rewbuffer']),
    locs['it'])
            self.writer.add_scalar('Train/mean_episode_length', statistics.mean(locs['lenbuffer']),
    locs['it'])
            self.writer.add_scalar('Train/mean_reward/time', statistics.mean(locs['rewbuffer']),
    self.tot_time)
            self.writer.add_scalar('Train/mean_episode_length/time',
    statistics.mean(locs['lenbuffer']), self.tot_time)
        str = f" \033[1m Learning iteration {locs['it']}/{self.current_learning_iteration +
    locs['num_learning_iterations']} \033[0m "
        if len(locs['rewbuffer']) > 0:
            log_string = (f"""{'#' * width}\n"""
                          f"""{str.center(width, ' ')}\n\n"""
                          f"""{'Computation:':>{pad}} {fps:.0f} steps/s (collection: {locs[
                            'collection_time']:.3f}s, learning {locs['learn_time']:.3f}s)\n"""
                          f"""{'Value function loss:':>{pad}} {locs['mean_value_loss']:.4f}\n"""
                          f"""{'Surrogate loss:':>{pad}} {locs['mean_surrogate_loss']:.4f}\n"""
                          f"""{'Mean action noise std:':>{pad}} {mean_std.item():.2f}\n"""
```

```
                        f"""{'Mean reward:':>{pad}} {statistics.mean(locs['rewbuffer']):.2f}\n"""
                        f"""{'Mean episode length:':>{pad}}
    {statistics.mean(locs['lenbuffer']):.2f}\n""")
        else:
            log_string = (f"""{'#' * width}\n"""
                        f"""{str.center(width, ' ')}\n\n"""
                        f"""{'Computation:':>{pad}} {fps:.0f} steps/s (collection: {locs[
                          'collection_time']:.3f}s, learning {locs['learn_time']:.3f}s)\n"""
                        f"""{'Value function loss:':>{pad}} {locs['mean_value_loss']:.4f}\n"""
                        f"""{'Surrogate loss:':>{pad}} {locs['mean_surrogate_loss']:.4f}\n"""
                        f"""{'Mean action noise std:':>{pad}} {mean_std.item():.2f}\n""")
        log_string += ep_string
        log_string += (f"""{'-' * width}\n"""
                       f"""{'Total timesteps:':>{pad}} {self.tot_timesteps}\n"""
                       f"""{'Iteration time:':>{pad}} {iteration_time:.2f}s\n"""
                       f"""{'Total time:':>{pad}} {self.tot_time:.2f}s\n"""
                       f"""{'ETA:':>{pad}} {self.tot_time / (locs['it'] + 1) * (
                              locs['num_learning_iterations'] - locs['it']):.1f}s\n""")
        print(log_string)
    def save(self, path, infos=None):
        torch.save({
            'model_state_dict': self.alg.actor_critic.state_dict(),
            'optimizer_state_dict': self.alg.optimizer.state_dict(),
            'iter': self.current_learning_iteration,
            'infos': infos,
            }, path)
    def load(self, path, load_optimizer=True):
        loaded_dict = torch.load(path)
        self.alg.actor_critic.load_state_dict(loaded_dict['model_state_dict'])
        if load_optimizer:
            self.alg.optimizer.load_state_dict(loaded_dict['optimizer_state_dict'])
        self.current_learning_iteration = loaded_dict['iter']
        return loaded_dict['infos']
    def get_inference_policy(self, device=None):
        self.alg.actor_critic.eval()
        if device is not None:
            self.alg.actor_critic.to(device)
        return self.alg.actor_critic.act_inference
import torch
def split_and_pad_trajectories(tensor, dones):
    """ Splits trajectories at done indices. Then concatenates them and padds with zeros up to the
    length og the longest trajectory.
    Returns masks corresponding to valid parts of the trajectories
    Example:
        Input: [ [a1, a2, a3, a4 | a5, a6],
                 [b1, b2 | b3, b4, b5 | b6]
                ]
        Output:[ [a1, a2, a3, a4], | [  [True, True, True, True],
                 [a5, a6, 0, 0],   |    [True, True, False, False],
                 [b1, b2, 0, 0],   |    [True, True, False, False],
```

```
                [b3, b4, b5, 0],  |    [True, True, True, False],
                [b6, 0, 0, 0]     |    [True, False, False, False],
               ]                  | ]
    Assumes that the inputy has the following dimension order: [time, number of envs, aditional
    dimensions]
    """
    dones = dones.clone()
    dones[-1] = 1
    flat_dones = dones.transpose(1, 0).reshape(-1, 1)
    done_indices = torch.cat((flat_dones.new_tensor([-1], dtype=torch.int64),
    flat_dones.nonzero()[:, 0]))
    trajectory_lengths = done_indices[1:] - done_indices[:-1]
    trajectory_lengths_list = trajectory_lengths.tolist()
    trajectories = torch.split(tensor.transpose(1, 0).flatten(0, 1),trajectory_lengths_list)
    padded_trajectories = torch.nn.utils.rnn.pad_sequence(trajectories)
    trajectory_masks = trajectory_lengths > torch.arange(0, tensor.shape[0],
    device=tensor.device).unsqueeze(1)
    return padded_trajectories, trajectory_masks
def unpad_trajectories(trajectories, masks):
    """ Does the inverse operation of  split_and_pad_trajectories()
    """
    return trajectories.transpose(1, 0)[masks.transpose(1, 0)].view(-1, trajectories.shape[0],
    trajectories.shape[-1]).transpose(1, 0)
from legged_gym.envs.base.legged_robot_config import LeggedRobotCfg, LeggedRobotCfgPPO
class GO2RoughCfg( LeggedRobotCfg ):
    class env:
        num_envs = 4096
        num_observations = 45
        num_privileged_obs = None
        num_actions = 12
        env_spacing = 3.
        send_timeouts = True
        episode_length_s = 20
        test = False
    class init_state( LeggedRobotCfg.init_state ):
        pos = [0.0, 0.0, 0.42]
        default_joint_angles = {
            'FL_hip_joint': 0.1,
            'FL_thigh_joint': 0.8,
            'FL_calf_joint': -1.5,
            'FR_hip_joint': -0.1 ,
            'FR_thigh_joint': 0.8,
            'FR_calf_joint': -1.5,
            'RL_hip_joint': 0.1,
            'RL_thigh_joint': 1.,
            'RL_calf_joint': -1.5,
            'RR_hip_joint': -0.1,
            'RR_thigh_joint': 1.,
            'RR_calf_joint': -1.5,
        }
```

```
    class rewards_ext:
        unsafe_spheres_pos = [
            [-2.66, -3.66, 0.4],
            [ 4.00, -3.00, 0.4],
            [-3.66,  2.33, 0.4],
            [ 3.00,  4.33, 0.4],
            [ 0.33, -4.00, 0.4],
            [-4.33, -0.33, 0.4],
            [ 2.66,  0.33, 0.4],
            [-0.66,  3.33, 0.4],
            [-2.00,  2.00, 0.4],
            [-2.00,  1.00, 0.4],
            [5.00,  0.00, 0.4],
            [2.00,  -2.00, 0.4],
            [-0.5,  -2.5, 0.4],
            [-4.00,  -2.00, 0.4],
        ]
        unsafe_sphere_radius = 0.25
        boundary_margin = 0.25
        target_sphere_pos = [0.0, 0.0, 0.4]
        target_sphere_radius = 0.5
        unsafe_radius_h_eval_scale = 2.0
    class terrain(LeggedRobotCfg.terrain ):
        mesh_type = 'plane'
        horizontal_scale = 0.1
        vertical_scale = 0.005
        border_size = 25
        curriculum = False
        static_friction = 1.0
        dynamic_friction = 1.0
        restitution = 0.
        measure_heights = False
        measured_points_x = [-0.8, -0.7, -0.6, -0.5, -0.4, -0.3, -0.2, -0.1, 0., 0.1, 0.2, 0.3,
    0.4, 0.5, 0.6, 0.7, 0.8]
        measured_points_y = [-0.5, -0.4, -0.3, -0.2, -0.1, 0., 0.1, 0.2, 0.3, 0.4, 0.5]
        selected = False
        terrain_kwargs = None
        max_init_terrain_level = 5
        terrain_length = 12.
        terrain_width = 12.
        num_rows= 20
        num_cols = 20
        terrain_proportions = [0.0, 0.6, 0.2, 0.2, 0.0]
        slope_treshold = 0.75
    class control( LeggedRobotCfg.control ):
        control_type = 'P'
        stiffness = {'joint': 20.}
        damping = {'joint': 0.5}
        action_scale = 0.25
        decimation = 4
```

```
    class asset( LeggedRobotCfg.asset ):
        file = '{LEGGED_GYM_ROOT_DIR}/resources/robots/go2/urdf/go2.urdf'
        name = "go2"
        foot_name = "foot"
        penalize_contacts_on = ["base","thigh", "calf"]
        terminate_after_contacts_on = ["base"]
        self_collisions = 1
    class domain_rand:
        randomize_friction = True
        friction_range = [0.5, 1.25]
        randomize_base_mass = False
        added_mass_range = [-0.5, 0.5]
        push_robots = True
        push_interval_s = 15
        max_push_vel_xy = 1.
    class rewards( LeggedRobotCfg.rewards ):
        soft_dof_pos_limit = 0.9
        base_height_target = 0.35
        touch_thr= 8
        command_dead = 0.1
        class scales( LeggedRobotCfg.rewards.scales ):
            termination = -0.0
            tracking_lin_vel = 1.0
            tracking_ang_vel = 0.4
            lin_vel_z = -2.0
            orientation = -0.
            torques = -0.00001
            dof_vel = -0.
            dof_acc = -2.5e-7
            base_height = -0.3
            feet_air_time =  1.0
            collision = -1.
            feet_stumble = -0.0
            action_rate = -0.01
            stand_still = -0.
            dof_pos_limits = -10.0
            hip_pos = -0.5
            orientation_eular = 0.32
            feet_contact_forces = -0.01
            foot_slip = -0.05
            vel_mismatch_exp = 0.4
            no_fly  = 0.05
class GO2RoughCfgPPO( LeggedRobotCfgPPO ):
    class algorithm( LeggedRobotCfgPPO.algorithm ):
        entropy_coef = 0.01
    class runner( LeggedRobotCfgPPO.runner ):
        run_name = ''
        max_iterations = 16000
        experiment_name = 'rough_go2'
        save_interval = 500
```

```
        resume = True
        resume_path =
    "/home/wutr/IsaacGym/legged_gym_go2/logs/rough_go2/Sep08_11-57-26_/model_18500.pt"
class GO2HighLevelCfg(GO2RoughCfg):
    """高层导航策略配置"""
    seed = 1
    enable_manual_lidar = True
    lidar_max_range = 8.0
    lidar_num_bins = 16
    target_lidar_num_bins = 16
    target_lidar_max_range = 8.0
    class env(GO2RoughCfg.env):
        num_observations = 7
        num_actions = 3
        high_level_action_repeat = 5
class GO2HighLevelCfgPPO(LeggedRobotCfgPPO):
    """高层导航策略PPO配置"""
    class algorithm(LeggedRobotCfgPPO.algorithm):
        entropy_coef = 0.01
        learning_rate = 1e-3
        num_learning_epochs = 10
        num_mini_batches = 8
        num_steps_per_env = 200
    class runner(LeggedRobotCfgPPO.runner):
        run_name = ''
        experiment_name = 'high_level_go2'
        max_iterations = 1500
        save_interval = 100
        resume = False
        resume_path =
    "/home/caohy/repositories/MCRA_RL/legged_gym_go2/legged_gym/scripts/logs/high_level_go2/20260105
    -102613/model_1300.pt"
        low_level_model_path =
    "/home/caohy/repositories/Go2HierarchicalReachAvoidRL/logs/rough_go2/Sep08_11-57-26_/model_18500
    .pt"
_base_high_level_obs = 8
_target_dim = GO2HighLevelCfg.target_lidar_num_bins
_lidar_dim = GO2HighLevelCfg.lidar_num_bins if GO2HighLevelCfg.enable_manual_lidar else 0
GO2HighLevelCfg.env.num_observations = _base_high_level_obs + _target_dim + _lidar_dim
from legged_gym.envs.base.legged_robot import LeggedRobot
import numpy as np
import math
from isaacgym.torch_utils import *
from isaacgym import gymtorch, gymapi, gymutil
import torch
class GO2Robot(LeggedRobot):
    def reset(self):
        """ Reset all robots"""
        self.reset_idx(torch.arange(self.num_envs, device=self.device))
        obs, privileged_obs, _, _, _ , _, _= self.step(torch.zeros(self.num_envs, self.num_actions,
```

```
    device=self.device, requires_grad=False))
        return obs, privileged_obs
    def step(self, actions):
        """
        Overrides the base class step to add custom metrics to the return values.
        """
        obs, privileged_obs, rews, dones, infos = super().step(actions)
        self._compute_safety_metrics()
        return obs, privileged_obs, rews, dones, infos, self.avoid_metric, self.reach_metric
    def compute_observations(self):
        """ Computes observations
        """
        self.obs_buf = torch.cat((
                                    self.base_ang_vel  * self.obs_scales.ang_vel,
                                    self.projected_gravity,
                                    self.commands[:, :3] * self.commands_scale,
                                    (self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos,
                                    self.dof_vel * self.obs_scales.dof_vel,
                                    self.actions
                                    ),dim=-1)
        if self.add_noise:
            self.obs_buf += (2 * torch.rand_like(self.obs_buf) - 1) * self.noise_scale_vec
    def _init_buffers(self):
        super()._init_buffers()
        self.avoid_metric = torch.zeros(self.num_envs, device=self.device, dtype=torch.float)
        self.reach_metric = torch.zeros(self.num_envs, device=self.device, dtype=torch.float)
        rigid_body_state_tensor = self.gym.acquire_rigid_body_state_tensor(self.sim)
        self.rigid_body_states = gymtorch.wrap_tensor(rigid_body_state_tensor).view(self.num_envs,
    -1, 13)
    def _reset_root_states(self, env_ids):
        """Overrides base reset to sample initial xy within the full 6x6 arena."""
        if len(env_ids) == 0:
            return
        super()._reset_root_states(env_ids)
        terrain_length = getattr(self.cfg.terrain, "terrain_length", 6.0)
        terrain_width = getattr(self.cfg.terrain, "terrain_width", 6.0)
        spawn_range_x = max(terrain_length * 0.5, 0.0)
        spawn_range_y = max(terrain_width * 0.5, 0.0)
        random_x = torch_rand_float(-spawn_range_x, spawn_range_x, (len(env_ids), 1),
    device=self.device)
        random_y = torch_rand_float(-spawn_range_y, spawn_range_y, (len(env_ids), 1),
    device=self.device)
        self.root_states[env_ids, 0] = random_x.squeeze(-1) + self.env_origins[env_ids, 0]
        self.root_states[env_ids, 1] = random_y.squeeze(-1) + self.env_origins[env_ids, 1]
        random_yaw = torch_rand_float(-math.pi, math.pi, (len(env_ids), 1),
    device=self.device).squeeze(-1)
        half_yaw = 0.5 * random_yaw
        delta_quat = torch.zeros(len(env_ids), 4, device=self.device)
        delta_quat[:, 2] = torch.sin(half_yaw)
        delta_quat[:, 3] = torch.cos(half_yaw)
```

```
        base_quat = self.base_init_state[3:7].unsqueeze(0).repeat(len(env_ids), 1)
        randomized_quat = quat_mul(delta_quat, base_quat)
        self.root_states[env_ids, 3:7] = randomized_quat
        env_ids_int32 = env_ids.to(dtype=torch.int32)
        self.gym.set_actor_root_state_tensor_indexed(
            self.sim,
            gymtorch.unwrap_tensor(self.root_states),
            gymtorch.unwrap_tensor(env_ids_int32),
            len(env_ids_int32),
        )
    def _get_noise_scale_vec(self, cfg):
        """ Sets a vector used to scale the noise added to the observations.
            [NOTE]: Must be adapted when changing the observations structure
        Args:
            cfg (Dict): Environment config file
        Returns:
            [torch.Tensor]: Vector of scales used to multiply a uniform distribution in [-1, 1]
        """
        noise_vec = torch.zeros_like(self.obs_buf[0])
        self.add_noise = self.cfg.noise.add_noise
        noise_scales = self.cfg.noise.noise_scales
        noise_level = self.cfg.noise.noise_level
        noise_vec[:3] = noise_scales.ang_vel * noise_level * self.obs_scales.ang_vel
        noise_vec[3:6] = noise_scales.gravity * noise_level
        noise_vec[6:9] = 0.
        noise_vec[9:9+self.num_actions] = noise_scales.dof_pos * noise_level *
    self.obs_scales.dof_pos
        noise_vec[9+self.num_actions:9+2*self.num_actions] = noise_scales.dof_vel * noise_level *
    self.obs_scales.dof_vel
        noise_vec[9+2*self.num_actions:9+3*self.num_actions] = 0.
        return noise_vec
    def _reward_feet_contact_forces(self):
        return torch.sum((torch.norm(self.contact_forces[:, self.feet_indices, :], dim=-1) -
    self.cfg.rewards.max_contact_force).clip(min=0.), dim=1)
    def _reward_hip_pos(self):
        return (0.8-torch.abs(self.commands[:,1])) *  torch.sum(torch.square(self.dof_pos[:, [0, 3,
    6, 9]] - self.default_dof_pos[:, [0, 3, 6, 9]]), dim=1)
    def _reward_feet_contact_number(self):
        """
        Calculates a reward based on the number of feet contacts aligning with the gait phase.
        Rewards or penalizes depending on whether the foot contact matches the expected gait phase.
        """
        contact = self.contact_forces[:, self.feet_indices, 2] > self.cfg.rewards.touch_thr
        contact_num= torch.sum(contact,dim=1)
        reward = torch.where(contact_num <=2, 0, 1)
        return reward* (torch.norm(self.commands[:, :2], dim=1) > 0.1)
    def _reward_orientation_eular(self):
        """
        Calculates the reward for maintaining a flat base orientation. It penalizes deviation
        from the desired base orientation using the base euler angles and the projected gravity
```

```
    vector.
        """
        orientation = torch.exp(-torch.norm(self.projected_gravity[:, :2], dim=1) * 20)
        return orientation
    def _reward_feet_contact_forces(self):
        """
        Calculates the reward for keeping contact forces within a specified range. Penalizes
        high contact forces on the feet.
        """
        return torch.sum((torch.norm(self.contact_forces[:, self.feet_indices, :], dim=-1) -
self.cfg.rewards.max_contact_force).clip(0, 100), dim=1)
    def _reward_foot_slip(self):
        """
        Calculates the reward for minimizing foot slip. The reward is based on the contact forces
        and the speed of the feet. A contact threshold is used to determine if the foot is in
contact
        with the ground. The speed of the foot is calculated and scaled by the contact condition.
        """
        contact = self.contact_forces[:, self.feet_indices, 2] > self.cfg.rewards.touch_thr
        foot_speed_norm = torch.norm(self.rigid_body_states[:, self.feet_indices, 10:12], dim=2)
        rew = torch.sqrt(foot_speed_norm)
        rew *= contact
        return torch.sum(rew, dim=1)
    def _reward_vel_mismatch_exp(self):
        """
        Computes a reward based on the mismatch in the robot's linear and angular velocities.
        Encourages the robot to maintain a stable velocity by penalizing large deviations.
        """
        lin_mismatch = torch.exp(-torch.square(self.base_lin_vel[:, 2]) * 10)
        ang_mismatch = torch.exp(-torch.norm(self.base_ang_vel[:, :2], dim=1) * 5.)
        c_update = (lin_mismatch + ang_mismatch) / 2.
        return c_update
    def _reward_no_fly(self):
        contacts = self.contact_forces[:, self.feet_indices, 2] > self.cfg.rewards.touch_thr
        double_contact = torch.sum(1.*contacts, dim=1)==2
        return double_contact*(torch.norm(self.commands[:, :2], dim=1) >
self.cfg.rewards.command_dead)
    def _compute_safety_metrics(self):
        """ Computes the avoid and reach metrics and stores them on self. """
        radius_scale = getattr(self.cfg.rewards_ext, "unsafe_radius_h_eval_scale", 1.0)
        effective_radius = self.cfg.rewards_ext.unsafe_sphere_radius * radius_scale
        if hasattr(self.cfg.rewards_ext, 'unsafe_spheres_pos') and
len(self.cfg.rewards_ext.unsafe_spheres_pos) > 0:
            unsafe_spheres_pos_base = torch.tensor(
                self.cfg.rewards_ext.unsafe_spheres_pos,
                device=self.device,
                dtype=torch.float
            )
            num_obstacles = unsafe_spheres_pos_base.shape[0]
            unsafe_spheres_pos = self.env_origins.unsqueeze(1) +
```

```
    unsafe_spheres_pos_base.unsqueeze(0)
            dist_to_unsafe_all = torch.norm(
                self.base_pos.unsqueeze(1) - unsafe_spheres_pos,
                dim=2
            )
            dist_to_unsafe = torch.min(dist_to_unsafe_all, dim=1)[0]
        else:
            unsafe_pos_base = torch.tensor(
                self.cfg.rewards_ext.unsafe_sphere_pos,
                device=self.device,
                dtype=torch.float
            )
            unsafe_pos = unsafe_pos_base.unsqueeze(0) + self.env_origins
            dist_to_unsafe = torch.norm(self.base_pos - unsafe_pos, dim=1)
        avoid_metric = torch.clamp(effective_radius - dist_to_unsafe, min=0.)
        terrain_length = getattr(self.cfg.terrain, "terrain_length", 6.0)
        terrain_width = getattr(self.cfg.terrain, "terrain_width", 6.0)
        half_length = max(terrain_length * 0.5, 0.0)
        half_width = max(terrain_width * 0.5, 0.0)
        if half_length > 0.0 and half_width > 0.0:
            rel_pos_xy = self.base_pos[:, :2] - self.env_origins[:, :2]
            gap_x = half_length - torch.abs(rel_pos_xy[:, 0])
            gap_y = half_width - torch.abs(rel_pos_xy[:, 1])
            boundary_distance = torch.min(gap_x, gap_y)
            boundary_margin = getattr(self.cfg.rewards_ext, "boundary_margin", 0.25)
            boundary_metric = torch.clamp(boundary_margin - boundary_distance, min=0.0)
            avoid_metric = torch.maximum(avoid_metric, boundary_metric)
            outside = boundary_distance < 0.0
            if outside.any():
                self.reset_buf[outside] = 1
        target_pos_base = torch.tensor(self.cfg.rewards_ext.target_sphere_pos, device=self.device,
    dtype=torch.float)
        target_pos = target_pos_base.unsqueeze(0) + self.env_origins
        base_pos_xy = self.base_pos[:, :2]
        target_pos_xy = target_pos[:, :2]
        reach_metric = torch.norm(base_pos_xy - target_pos_xy, dim=1)
        self.avoid_metric = avoid_metric
        self.reach_metric = reach_metric
import torch
import numpy as np
import math
from legged_gym.envs.go2.go2_env import GO2Robot
from legged_gym.utils.math import quat_apply
from legged_gym.utils.helpers import class_to_dict
class HighLevelNavigationEnv:
    """
    High-level navigation wrapper.
    - Observations: [cos_heading, sin_heading, body vx, body vy, yaw_rate, dist_to_target,
    target_dir_body]
      plus optional lidar bins
```

```
    - Actions: [vx, vy, vyaw] velocity commands
    - Invokes the low-level GO2 environment to execute motion
    """
    def __init__(self, base_env: GO2Robot, cfg):
        self.base_env = base_env
        self.cfg = cfg
        self.device = base_env.device
        self.num_envs = base_env.num_envs
        self.forward_vec = torch.tensor([1., 0., 0.], device=self.device, dtype=torch.float)
        unsafe_positions = getattr(self.cfg, "unsafe_spheres_pos", None)
        if unsafe_positions is not None and len(unsafe_positions) > 0:
            unsafe_positions = torch.tensor(unsafe_positions, device=self.device, dtype=torch.float)
        else:
            unsafe_positions = torch.tensor(
                [self.cfg.unsafe_sphere_pos], device=self.device, dtype=torch.float
            )
        self._unsafe_pos_base = unsafe_positions
        self.num_obstacles = self._unsafe_pos_base.shape[0]
        self._target_pos_base = torch.tensor(
            self.cfg.target_sphere_pos, device=self.device, dtype=torch.float
        )
        self._base_obs_dim = 8
        self.use_manual_lidar = getattr(self.cfg, "enable_manual_lidar", True)
        self.lidar_num_bins = getattr(self.cfg, "lidar_num_bins", 16)
        self.target_lidar_num_bins = getattr(self.cfg, "target_lidar_num_bins", self.lidar_num_bins)
        self.target_lidar_max_range = getattr(self.cfg, "target_lidar_max_range", getattr(self.cfg,
    "lidar_max_range", 10.0))
        lidar_feature_dim = self.lidar_num_bins if self.use_manual_lidar else 0
        target_feature_dim = self.target_lidar_num_bins if self.target_lidar_num_bins > 0 else 0
        self.num_high_level_obs = self._base_obs_dim + target_feature_dim + lidar_feature_dim
        self.num_high_level_actions = 3
        boundary_extents = getattr(self.cfg, "boundary_half_extents", (3.0, 3.0))
        self.boundary_half_extents = torch.tensor(boundary_extents, device=self.device,
    dtype=torch.float)
        self.boundary_margin = getattr(self.cfg, "boundary_margin", 0.25)
        if self.use_manual_lidar and self.lidar_num_bins > 0:
            bin_size = 2 * math.pi / float(self.lidar_num_bins)
            angles = torch.arange(self.lidar_num_bins, device=self.device, dtype=torch.float) *
    bin_size - math.pi + 0.5 * bin_size
            self._lidar_dir_body = torch.stack((torch.cos(angles), torch.sin(angles)), dim=-1)
        else:
            self._lidar_dir_body = None
        self.high_level_obs_buf = torch.zeros(
            self.num_envs, self.num_high_level_obs,
            dtype=torch.float, device=self.device
        )
        self.action_scale = getattr(cfg, 'action_scale', [1.0, 1.0, 1.0])
        self.action_scale = torch.tensor(self.action_scale, device=self.device, dtype=torch.float)
    def reset(self):
        """
```

```
        Reset the environment and return initial observations plus g/h values.
        Returns:
            high_level_obs: [num_envs, 7] 高层观测（含目标/危险区距离）
            g_values: [num_envs] initial g-function values
            h_values: [num_envs] initial h-function values
        """
        base_obs = self.base_env.reset()
        self._compute_high_level_observations()
        initial_avoid_metric, initial_reach_metric = self._get_current_metrics()
        initial_g_values = self._compute_g_function(initial_reach_metric)
        initial_h_values = self._compute_h_function(initial_avoid_metric)
        return self.high_level_obs_buf, initial_g_values, initial_h_values
    def update_velocity_commands(self, high_level_actions):
        """
        将高层动作转换为速度命令并更新到底层环境
        Args:
            high_level_actions: [num_envs, 3] 速度命令 [vx, vy, vyaw]
        """
        high_level_actions = torch.clip(high_level_actions, -1.0, 1.0)
        velocity_commands = high_level_actions * self.action_scale
        self.base_env.commands[:, 0] = velocity_commands[:, 0] * 0.6
        self.base_env.commands[:, 1] = velocity_commands[:, 1] * 0.2
        self.base_env.commands[:, 2] = velocity_commands[:, 2] * 0.8
    def compute_g_h_values(self, avoid_metric, reach_metric):
        """
        Compute g/h values based on safety and reach metrics
        Args:
            avoid_metric: 避障指标
            reach_metric: 到达指标
        Returns:
            g_values: [num_envs] g-function values
            h_values: [num_envs] h-function values
        """
        g_values = self._compute_g_function(reach_metric)
        h_values = self._compute_h_function(avoid_metric)
        return g_values, h_values
    def _compute_high_level_observations(self):
        """Compute high-level observations including target/unsafe distances."""
        robot_pos = self.base_env.base_pos
        forward_vec_expanded = self.forward_vec.unsqueeze(0).expand(self.num_envs, -1)
        forward = quat_apply(self.base_env.base_quat, forward_vec_expanded)
        heading_cos = forward[:, 0]
        heading_sin = forward[:, 1]
        self.high_level_obs_buf[:, 0] = heading_cos
        self.high_level_obs_buf[:, 1] = heading_sin
        base_lin_vel_body = self.base_env.base_lin_vel
        lin_vel_scale = 2.0
        self.high_level_obs_buf[:, 2] = torch.clamp(base_lin_vel_body[:, 0] * lin_vel_scale, -1.0,
    1.0)
        self.high_level_obs_buf[:, 3] = torch.clamp(base_lin_vel_body[:, 1] * lin_vel_scale, -1.0,
```

```
        self.rew_log.clear()
    def print_rewards(self):
        print("Average rewards per second:")
        for key, values in self.rew_log.items():
            mean = np.sum(np.array(values)) / self.num_episodes
            print(f" - {key}: {mean}")
        print(f"Total number of episodes: {self.num_episodes}")
    def __del__(self):
        if self.plot_process is not None:
            self.plot_process.kill()
import os
import numpy as np
import random
import torch
@torch.jit.script
def copysign(a, b):
    a = torch.tensor(a, device=b.device, dtype=torch.float).repeat(b.shape[0])
    return torch.abs(a) * torch.sign(b)
def get_euler_xyz(q):
    qx, qy, qz, qw = 0, 1, 2, 3
    sinr_cosp = 2.0 * (q[:, qw] * q[:, qx] + q[:, qy] * q[:, qz])
    cosr_cosp = q[:, qw] * q[:, qw] - q[:, qx] * \
                q[:, qx] - q[:, qy] * q[:, qy] + q[:, qz] * q[:, qz]
    roll = torch.atan2(sinr_cosp, cosr_cosp)
    sinp = 2.0 * (q[:, qw] * q[:, qy] - q[:, qz] * q[:, qx])
    pitch = torch.where(
        torch.abs(sinp) >= 1, copysign(np.pi / 2.0, sinp), torch.asin(sinp))
    siny_cosp = 2.0 * (q[:, qw] * q[:, qz] + q[:, qx] * q[:, qy])
    cosy_cosp = q[:, qw] * q[:, qw] + q[:, qx] * \
                q[:, qx] - q[:, qy] * q[:, qy] - q[:, qz] * q[:, qz]
    yaw = torch.atan2(siny_cosp, cosy_cosp)
    return torch.stack((roll, pitch, yaw), dim=-1)
import torch
from torch import Tensor
import numpy as np
from isaacgym.torch_utils import quat_apply, normalize
from typing import Tuple
def quat_apply_yaw(quat, vec):
    quat_yaw = quat.clone().view(-1, 4)
    quat_yaw[:, :2] = 0.
    quat_yaw = normalize(quat_yaw)
    return quat_apply(quat_yaw, vec)
def wrap_to_pi(angles):
    angles %= 2*np.pi
    angles -= 2*np.pi * (angles > np.pi)
    return angles
def torch_rand_sqrt_float(lower, upper, shape, device):
    r = 2*torch.rand(*shape, device=device) - 1
    r = torch.where(r<0., -torch.sqrt(-r), torch.sqrt(r))
    r =  (r + 1.) / 2.
```

```
    return (upper - lower) * r + lower
from legged_gym.envs.base.legged_robot_config import LeggedRobotCfg, LeggedRobotCfgPPO
class H1_2RoughCfg(LeggedRobotCfg):
    class init_state(LeggedRobotCfg.init_state):
        pos = [0.0, 0.0, 1.05]
        default_joint_angles = {
            'left_hip_yaw_joint': 0,
            'left_hip_roll_joint': 0,
            'left_hip_pitch_joint': -0.16,
            'left_knee_joint': 0.36,
            'left_ankle_pitch_joint': -0.2,
            'left_ankle_roll_joint': 0.0,
            'right_hip_yaw_joint': 0,
            'right_hip_roll_joint': 0,
            'right_hip_pitch_joint': -0.16,
            'right_knee_joint': 0.36,
            'right_ankle_pitch_joint': -0.2,
            'right_ankle_roll_joint': 0.0,
            'torso_joint': 0,
            'left_shoulder_pitch_joint': 0.4,
            'left_shoulder_roll_joint': 0,
            'left_shoulder_yaw_joint': 0,
            'left_elbow_pitch_joint': 0.3,
            'right_shoulder_pitch_joint': 0.4,
            'right_shoulder_roll_joint': 0,
            'right_shoulder_yaw_joint': 0,
            'right_elbow_pitch_joint': 0.3,
        }
    class env(LeggedRobotCfg.env):
        num_observations = 47
        num_privileged_obs = 50
        num_actions = 12
    class control(LeggedRobotCfg.control):
        control_type = 'P'
        stiffness = {
            'hip_yaw_joint': 200.,
            'hip_roll_joint': 200.,
            'hip_pitch_joint': 200.,
            'knee_joint': 300.,
            'ankle_pitch_joint': 40.,
            'ankle_roll_joint': 40.,
        }
        damping = {
            'hip_yaw_joint': 2.5,
            'hip_roll_joint': 2.5,
            'hip_pitch_joint': 2.5,
            'knee_joint': 4,
            'ankle_pitch_joint': 2.0,
            'ankle_roll_joint': 2.0,
        }
```

```
        action_scale = 0.25
        decimation = 8
    class sim(LeggedRobotCfg.sim):
        dt =  0.0025
    class domain_rand(LeggedRobotCfg.domain_rand):
        randomize_friction = True
        friction_range = [0.1, 1.25]
        randomize_base_mass = True
        added_mass_range = [-1., 3.]
        push_robots = True
        push_interval_s = 5
        max_push_vel_xy = 1.5
    class asset(LeggedRobotCfg.asset):
        file = '{LEGGED_GYM_ROOT_DIR}/resources/robots/h1_2/h1_2_12dof.urdf'
        name = "h1_2"
        foot_name = "ankle_roll"
        penalize_contacts_on = ["hip", "knee"]
        terminate_after_contacts_on = ["pelvis"]
        self_collisions = 0
        flip_visual_attachments = False
        armature = 1e-3
    class rewards(LeggedRobotCfg.rewards):
        soft_dof_pos_limit = 0.9
        base_height_target = 1.0
        class scales(LeggedRobotCfg.rewards.scales):
            tracking_lin_vel = 1.0
            tracking_ang_vel = 0.5
            lin_vel_z = -2.0
            ang_vel_xy = -0.05
            orientation = -1.0
            base_height = -10.0
            dof_acc = -2.5e-7
            dof_vel = -1e-3
            feet_air_time = 0.0
            collision = 0.0
            action_rate = -0.01
            dof_pos_limits = -5.0
            alive = 0.15
            hip_pos = -1.0
            contact_no_vel = -0.2
            feet_swing_height = -20.0
            contact = 0.18
class H1_2RoughCfgPPO(LeggedRobotCfgPPO):
    class policy:
        init_noise_std = 0.8
        actor_hidden_dims = [32]
        critic_hidden_dims = [32]
        activation = 'elu'
        rnn_type = 'lstm'
        rnn_hidden_size = 64
```

```
        rnn_num_layers = 1
    class algorithm( LeggedRobotCfgPPO.algorithm ):
        entropy_coef = 0.01
    class runner( LeggedRobotCfgPPO.runner ):
        policy_class_name = "ActorCriticRecurrent"
        max_iterations = 10000
        run_name = ''
        experiment_name = 'h1_2'
from legged_gym.envs.base.legged_robot import LeggedRobot
from isaacgym.torch_utils import *
from isaacgym import gymtorch, gymapi, gymutil
import torch
class H1_2Robot(LeggedRobot):
    def _get_noise_scale_vec(self, cfg):
        """ Sets a vector used to scale the noise added to the observations.
            [NOTE]: Must be adapted when changing the observations structure
        Args:
            cfg (Dict): Environment config file
        Returns:
            [torch.Tensor]: Vector of scales used to multiply a uniform distribution in [-1, 1]
        """
        noise_vec = torch.zeros_like(self.obs_buf[0])
        self.add_noise = self.cfg.noise.add_noise
        noise_scales = self.cfg.noise.noise_scales
        noise_level = self.cfg.noise.noise_level
        noise_vec[:3] = noise_scales.ang_vel * noise_level * self.obs_scales.ang_vel
        noise_vec[3:6] = noise_scales.gravity * noise_level
        noise_vec[6:9] = 0.
        noise_vec[9:9+self.num_actions] = noise_scales.dof_pos * noise_level *
    self.obs_scales.dof_pos
        noise_vec[9+self.num_actions:9+2*self.num_actions] = noise_scales.dof_vel * noise_level *
    self.obs_scales.dof_vel
        noise_vec[9+2*self.num_actions:9+3*self.num_actions] = 0.
        noise_vec[9+3*self.num_actions:9+3*self.num_actions+2] = 0.
        return noise_vec
    def _init_foot(self):
        self.feet_num = len(self.feet_indices)
        rigid_body_state = self.gym.acquire_rigid_body_state_tensor(self.sim)
        self.rigid_body_states = gymtorch.wrap_tensor(rigid_body_state)
        self.rigid_body_states_view = self.rigid_body_states.view(self.num_envs, -1, 13)
        self.feet_state = self.rigid_body_states_view[:, self.feet_indices, :]
        self.feet_pos = self.feet_state[:, :, :3]
        self.feet_vel = self.feet_state[:, :, 7:10]
    def _init_buffers(self):
        super()._init_buffers()
        self._init_foot()
    def update_feet_state(self):
        self.gym.refresh_rigid_body_state_tensor(self.sim)
        self.feet_state = self.rigid_body_states_view[:, self.feet_indices, :]
        self.feet_pos = self.feet_state[:, :, :3]
```

```
        self.feet_vel = self.feet_state[:, :, 7:10]
    def _post_physics_step_callback(self):
        self.update_feet_state()
        period = 0.8
        offset = 0.5
        self.phase = (self.episode_length_buf * self.dt) % period / period
        self.phase_left = self.phase
        self.phase_right = (self.phase + offset) % 1
        self.leg_phase = torch.cat([self.phase_left.unsqueeze(1), self.phase_right.unsqueeze(1)],
    dim=-1)
        return super()._post_physics_step_callback()
    def compute_observations(self):
        """ Computes observations
        """
        sin_phase = torch.sin(2 * np.pi * self.phase ).unsqueeze(1)
        cos_phase = torch.cos(2 * np.pi * self.phase ).unsqueeze(1)
        self.obs_buf = torch.cat((  self.base_ang_vel  * self.obs_scales.ang_vel,
                                    self.projected_gravity,
                                    self.commands[:, :3] * self.commands_scale,
                                    (self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos,
                                    self.dof_vel * self.obs_scales.dof_vel,
                                    self.actions,
                                    sin_phase,
                                    cos_phase
                                    ),dim=-1)
        self.privileged_obs_buf = torch.cat((  self.base_lin_vel * self.obs_scales.lin_vel,
                                    self.base_ang_vel  * self.obs_scales.ang_vel,
                                    self.projected_gravity,
                                    self.commands[:, :3] * self.commands_scale,
                                    (self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos,
                                    self.dof_vel * self.obs_scales.dof_vel,
                                    self.actions,
                                    sin_phase,
                                    cos_phase
                                    ),dim=-1)
        if self.add_noise:
            self.obs_buf += (2 * torch.rand_like(self.obs_buf) - 1) * self.noise_scale_vec
    def _reward_contact(self):
        res = torch.zeros(self.num_envs, dtype=torch.float, device=self.device)
        for i in range(self.feet_num):
            is_stance = self.leg_phase[:, i] < 0.55
            contact = self.contact_forces[:, self.feet_indices[i], 2] > 1
            res += ~(contact ^ is_stance)
        return res
    def _reward_feet_swing_height(self):
        contact = torch.norm(self.contact_forces[:, self.feet_indices, :3], dim=2) > 1.
        pos_error = torch.square(self.feet_pos[:, :, 2] - 0.08) * ~contact
        return torch.sum(pos_error, dim=(1))
    def _reward_alive(self):
        return 1.0
```

```
    def _reward_contact_no_vel(self):
        contact = torch.norm(self.contact_forces[:, self.feet_indices, :3], dim=2) > 1.
        contact_feet_vel = self.feet_vel * contact.unsqueeze(-1)
        penalize = torch.square(contact_feet_vel[:, :, :3])
        return torch.sum(penalize, dim=(1,2))
    def _reward_hip_pos(self):
        return torch.sum(torch.square(self.dof_pos[:,[0,2,6,8]]), dim=1)
from legged_gym.envs.base.legged_robot_config import LeggedRobotCfg, LeggedRobotCfgPPO
class H1RoughCfg( LeggedRobotCfg ):
    class init_state( LeggedRobotCfg.init_state ):
        pos = [0.0, 0.0, 1.0]
        default_joint_angles = {
           'left_hip_yaw_joint' : 0. ,
           'left_hip_roll_joint' : 0,
           'left_hip_pitch_joint' : -0.1,
           'left_knee_joint' : 0.3,
           'left_ankle_joint' : -0.2,
           'right_hip_yaw_joint' : 0.,
           'right_hip_roll_joint' : 0,
           'right_hip_pitch_joint' : -0.1,
           'right_knee_joint' : 0.3,
           'right_ankle_joint' : -0.2,
           'torso_joint' : 0.,
           'left_shoulder_pitch_joint' : 0.,
           'left_shoulder_roll_joint' : 0,
           'left_shoulder_yaw_joint' : 0.,
           'left_elbow_joint'  : 0.,
           'right_shoulder_pitch_joint' : 0.,
           'right_shoulder_roll_joint' : 0.0,
           'right_shoulder_yaw_joint' : 0.,
           'right_elbow_joint' : 0.,
        }
    class env(LeggedRobotCfg.env):
        num_observations = 41
        num_privileged_obs = 44
        num_actions = 10
    class domain_rand(LeggedRobotCfg.domain_rand):
        randomize_friction = True
        friction_range = [0.1, 1.25]
        randomize_base_mass = True
        added_mass_range = [-1., 3.]
        push_robots = True
        push_interval_s = 5
        max_push_vel_xy = 1.5
    class control( LeggedRobotCfg.control ):
        control_type = 'P'
        stiffness = {'hip_yaw': 150,
                     'hip_roll': 150,
                     'hip_pitch': 150,
                     'knee': 200,
```

```
                    'ankle': 40,
                    'torso': 300,
                    'shoulder': 150,
                    "elbow":100,
                    }
        damping = {  'hip_yaw': 2,
                    'hip_roll': 2,
                    'hip_pitch': 2,
                    'knee': 4,
                    'ankle': 2,
                    'torso': 6,
                    'shoulder': 2,
                    "elbow":2,
                    }
        action_scale = 0.25
        decimation = 4
    class asset( LeggedRobotCfg.asset ):
        file = '{LEGGED_GYM_ROOT_DIR}/resources/robots/h1/urdf/h1.urdf'
        name = "h1"
        foot_name = "ankle"
        penalize_contacts_on = ["hip", "knee"]
        terminate_after_contacts_on = ["pelvis"]
        self_collisions = 0
        flip_visual_attachments = False
    class rewards( LeggedRobotCfg.rewards ):
        soft_dof_pos_limit = 0.9
        base_height_target = 1.05
        class scales( LeggedRobotCfg.rewards.scales ):
            tracking_lin_vel = 1.0
            tracking_ang_vel = 0.5
            lin_vel_z = -2.0
            ang_vel_xy = -0.05
            orientation = -1.0
            base_height = -10.0
            dof_acc = -2.5e-7
            feet_air_time = 0.0
            collision = -1.0
            action_rate = -0.01
            torques = 0.0
            dof_pos_limits = -5.0
            alive = 0.15
            hip_pos = -1.0
            contact_no_vel = -0.2
            feet_swing_height = -20.0
            contact = 0.18
class H1RoughCfgPPO( LeggedRobotCfgPPO ):
    class policy:
        init_noise_std = 0.8
        actor_hidden_dims = [32]
        critic_hidden_dims = [32]
```

```
        activation = 'elu'
        rnn_type = 'lstm'
        rnn_hidden_size = 64
        rnn_num_layers = 1
    class algorithm( LeggedRobotCfgPPO.algorithm ):
        entropy_coef = 0.01
    class runner( LeggedRobotCfgPPO.runner ):
        policy_class_name = "ActorCriticRecurrent"
        max_iterations = 10000
        run_name = ''
        experiment_name = 'h1'
from legged_gym.envs.base.legged_robot import LeggedRobot
from isaacgym.torch_utils import *
from isaacgym import gymtorch, gymapi, gymutil
import torch
class H1Robot(LeggedRobot):
    def _get_noise_scale_vec(self, cfg):
        """ Sets a vector used to scale the noise added to the observations.
            [NOTE]: Must be adapted when changing the observations structure
        Args:
            cfg (Dict): Environment config file
        Returns:
            [torch.Tensor]: Vector of scales used to multiply a uniform distribution in [-1, 1]
        """
        noise_vec = torch.zeros_like(self.obs_buf[0])
        self.add_noise = self.cfg.noise.add_noise
        noise_scales = self.cfg.noise.noise_scales
        noise_level = self.cfg.noise.noise_level
        noise_vec[:3] = noise_scales.ang_vel * noise_level * self.obs_scales.ang_vel
        noise_vec[3:6] = noise_scales.gravity * noise_level
        noise_vec[6:9] = 0.
        noise_vec[9:9+self.num_actions] = noise_scales.dof_pos * noise_level *
    self.obs_scales.dof_pos
        noise_vec[9+self.num_actions:9+2*self.num_actions] = noise_scales.dof_vel * noise_level *
    self.obs_scales.dof_vel
        noise_vec[9+2*self.num_actions:9+3*self.num_actions] = 0.
        noise_vec[9+3*self.num_actions:9+3*self.num_actions+2] = 0.
        return noise_vec
    def _init_foot(self):
        self.feet_num = len(self.feet_indices)
        rigid_body_state = self.gym.acquire_rigid_body_state_tensor(self.sim)
        self.rigid_body_states = gymtorch.wrap_tensor(rigid_body_state)
        self.rigid_body_states_view = self.rigid_body_states.view(self.num_envs, -1, 13)
        self.feet_state = self.rigid_body_states_view[:, self.feet_indices, :]
        self.feet_pos = self.feet_state[:, :, :3]
        self.feet_vel = self.feet_state[:, :, 7:10]
    def _init_buffers(self):
        super()._init_buffers()
        self._init_foot()
    def update_feet_state(self):
```

```
        self.gym.refresh_rigid_body_state_tensor(self.sim)
        self.feet_state = self.rigid_body_states_view[:, self.feet_indices, :]
        self.feet_pos = self.feet_state[:, :, :3]
        self.feet_vel = self.feet_state[:, :, 7:10]
    def _post_physics_step_callback(self):
        self.update_feet_state()
        period = 0.8
        offset = 0.5
        self.phase = (self.episode_length_buf * self.dt) % period / period
        self.phase_left = self.phase
        self.phase_right = (self.phase + offset) % 1
        self.leg_phase = torch.cat([self.phase_left.unsqueeze(1), self.phase_right.unsqueeze(1)],
    dim=-1)
        return super()._post_physics_step_callback()
    def compute_observations(self):
        """ Computes observations
        """
        sin_phase = torch.sin(2 * np.pi * self.phase ).unsqueeze(1)
        cos_phase = torch.cos(2 * np.pi * self.phase ).unsqueeze(1)
        self.obs_buf = torch.cat((  self.base_ang_vel  * self.obs_scales.ang_vel,
                                    self.projected_gravity,
                                    self.commands[:, :3] * self.commands_scale,
                                    (self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos,
                                    self.dof_vel * self.obs_scales.dof_vel,
                                    self.actions,
                                    sin_phase,
                                    cos_phase
                                    ),dim=-1)
        self.privileged_obs_buf = torch.cat((  self.base_lin_vel * self.obs_scales.lin_vel,
                                    self.base_ang_vel  * self.obs_scales.ang_vel,
                                    self.projected_gravity,
                                    self.commands[:, :3] * self.commands_scale,
                                    (self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos,
                                    self.dof_vel * self.obs_scales.dof_vel,
                                    self.actions,
                                    sin_phase,
                                    cos_phase
                                    ),dim=-1)
        if self.add_noise:
            self.obs_buf += (2 * torch.rand_like(self.obs_buf) - 1) * self.noise_scale_vec
    def _reward_contact(self):
        res = torch.zeros(self.num_envs, dtype=torch.float, device=self.device)
        for i in range(self.feet_num):
            is_stance = self.leg_phase[:, i] < 0.55
            contact = self.contact_forces[:, self.feet_indices[i], 2] > 1
            res += ~(contact ^ is_stance)
        return res
    def _reward_feet_swing_height(self):
        contact = torch.norm(self.contact_forces[:, self.feet_indices, :3], dim=2) > 1.
        pos_error = torch.square(self.feet_pos[:, :, 2] - 0.08) * ~contact
```

```
        return torch.sum(pos_error, dim=(1))
    def _reward_alive(self):
        return 1.0
    def _reward_contact_no_vel(self):
        contact = torch.norm(self.contact_forces[:, self.feet_indices, :3], dim=2) > 1.
        contact_feet_vel = self.feet_vel * contact.unsqueeze(-1)
        penalize = torch.square(contact_feet_vel[:, :, :3])
        return torch.sum(penalize, dim=(1,2))
    def _reward_hip_pos(self):
        return torch.sum(torch.square(self.dof_pos[:,[0,1,5,6]]), dim=1)
from legged_gym.envs.base.legged_robot_config import LeggedRobotCfg, LeggedRobotCfgPPO
class G1RoughCfg( LeggedRobotCfg ):
    class init_state( LeggedRobotCfg.init_state ):
        pos = [0.0, 0.0, 0.8]
        default_joint_angles = {
           'left_hip_yaw_joint' : 0. ,
           'left_hip_roll_joint' : 0,
           'left_hip_pitch_joint' : -0.1,
           'left_knee_joint' : 0.3,
           'left_ankle_pitch_joint' : -0.2,
           'left_ankle_roll_joint' : 0,
           'right_hip_yaw_joint' : 0.,
           'right_hip_roll_joint' : 0,
           'right_hip_pitch_joint' : -0.1,
           'right_knee_joint' : 0.3,
           'right_ankle_pitch_joint': -0.2,
           'right_ankle_roll_joint' : 0,
           'torso_joint' : 0.
        }
    class env(LeggedRobotCfg.env):
        num_observations = 47
        num_privileged_obs = 50
        num_actions = 12
    class domain_rand(LeggedRobotCfg.domain_rand):
        randomize_friction = True
        friction_range = [0.1, 1.25]
        randomize_base_mass = True
        added_mass_range = [-1., 3.]
        push_robots = True
        push_interval_s = 5
        max_push_vel_xy = 1.5
    class control( LeggedRobotCfg.control ):
        control_type = 'P'
        stiffness = {'hip_yaw': 100,
                     'hip_roll': 100,
                     'hip_pitch': 100,
                     'knee': 150,
                     'ankle': 40,
                     }
        damping = {  'hip_yaw': 2,
```

```
                    'hip_roll': 2,
                    'hip_pitch': 2,
                    'knee': 4,
                    'ankle': 2,
                    }
        action_scale = 0.25
        decimation = 4
    class asset( LeggedRobotCfg.asset ):
        file = '{LEGGED_GYM_ROOT_DIR}/resources/robots/g1_description/g1_12dof.urdf'
        name = "g1"
        foot_name = "ankle_roll"
        penalize_contacts_on = ["hip", "knee"]
        terminate_after_contacts_on = ["pelvis"]
        self_collisions = 0
        flip_visual_attachments = False
    class rewards( LeggedRobotCfg.rewards ):
        soft_dof_pos_limit = 0.9
        base_height_target = 0.78
        class scales( LeggedRobotCfg.rewards.scales ):
            tracking_lin_vel = 1.0
            tracking_ang_vel = 0.5
            lin_vel_z = -2.0
            ang_vel_xy = -0.05
            orientation = -1.0
            base_height = -10.0
            dof_acc = -2.5e-7
            dof_vel = -1e-3
            feet_air_time = 0.0
            collision = 0.0
            action_rate = -0.01
            dof_pos_limits = -5.0
            alive = 0.15
            hip_pos = -1.0
            contact_no_vel = -0.2
            feet_swing_height = -20.0
            contact = 0.18
class G1RoughCfgPPO( LeggedRobotCfgPPO ):
    class policy:
        init_noise_std = 0.8
        actor_hidden_dims = [32]
        critic_hidden_dims = [32]
        activation = 'elu'
        rnn_type = 'lstm'
        rnn_hidden_size = 64
        rnn_num_layers = 1
    class algorithm( LeggedRobotCfgPPO.algorithm ):
        entropy_coef = 0.01
    class runner( LeggedRobotCfgPPO.runner ):
        policy_class_name = "ActorCriticRecurrent"
        max_iterations = 10000
```

```
        run_name = ''
        experiment_name = 'g1'
from legged_gym.envs.base.legged_robot import LeggedRobot
from isaacgym.torch_utils import *
from isaacgym import gymtorch, gymapi, gymutil
import torch
class G1Robot(LeggedRobot):
    def _get_noise_scale_vec(self, cfg):
        """ Sets a vector used to scale the noise added to the observations.
            [NOTE]: Must be adapted when changing the observations structure
        Args:
            cfg (Dict): Environment config file
        Returns:
            [torch.Tensor]: Vector of scales used to multiply a uniform distribution in [-1, 1]
        """
        noise_vec = torch.zeros_like(self.obs_buf[0])
        self.add_noise = self.cfg.noise.add_noise
        noise_scales = self.cfg.noise.noise_scales
        noise_level = self.cfg.noise.noise_level
        noise_vec[:3] = noise_scales.ang_vel * noise_level * self.obs_scales.ang_vel
        noise_vec[3:6] = noise_scales.gravity * noise_level
        noise_vec[6:9] = 0.
        noise_vec[9:9+self.num_actions] = noise_scales.dof_pos * noise_level *
    self.obs_scales.dof_pos
        noise_vec[9+self.num_actions:9+2*self.num_actions] = noise_scales.dof_vel * noise_level *
    self.obs_scales.dof_vel
        noise_vec[9+2*self.num_actions:9+3*self.num_actions] = 0.
        noise_vec[9+3*self.num_actions:9+3*self.num_actions+2] = 0.
        return noise_vec
    def _init_foot(self):
        self.feet_num = len(self.feet_indices)
        rigid_body_state = self.gym.acquire_rigid_body_state_tensor(self.sim)
        self.rigid_body_states = gymtorch.wrap_tensor(rigid_body_state)
        self.rigid_body_states_view = self.rigid_body_states.view(self.num_envs, -1, 13)
        self.feet_state = self.rigid_body_states_view[:, self.feet_indices, :]
        self.feet_pos = self.feet_state[:, :, :3]
        self.feet_vel = self.feet_state[:, :, 7:10]
    def _init_buffers(self):
        super()._init_buffers()
        self._init_foot()
    def update_feet_state(self):
        self.gym.refresh_rigid_body_state_tensor(self.sim)
        self.feet_state = self.rigid_body_states_view[:, self.feet_indices, :]
        self.feet_pos = self.feet_state[:, :, :3]
        self.feet_vel = self.feet_state[:, :, 7:10]
    def _post_physics_step_callback(self):
        self.update_feet_state()
        period = 0.8
        offset = 0.5
        self.phase = (self.episode_length_buf * self.dt) % period / period
```

```
        self.phase_left = self.phase
        self.phase_right = (self.phase + offset) % 1
        self.leg_phase = torch.cat([self.phase_left.unsqueeze(1), self.phase_right.unsqueeze(1)],
    dim=-1)
        return super()._post_physics_step_callback()
    def compute_observations(self):
        """ Computes observations
        """
        sin_phase = torch.sin(2 * np.pi * self.phase ).unsqueeze(1)
        cos_phase = torch.cos(2 * np.pi * self.phase ).unsqueeze(1)
        self.obs_buf = torch.cat((  self.base_ang_vel  * self.obs_scales.ang_vel,
                                    self.projected_gravity,
                                    self.commands[:, :3] * self.commands_scale,
                                    (self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos,
                                    self.dof_vel * self.obs_scales.dof_vel,
                                    self.actions,
                                    sin_phase,
                                    cos_phase
                                    ),dim=-1)
        self.privileged_obs_buf = torch.cat((  self.base_lin_vel * self.obs_scales.lin_vel,
                                    self.base_ang_vel  * self.obs_scales.ang_vel,
                                    self.projected_gravity,
                                    self.commands[:, :3] * self.commands_scale,
                                    (self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos,
                                    self.dof_vel * self.obs_scales.dof_vel,
                                    self.actions,
                                    sin_phase,
                                    cos_phase
                                    ),dim=-1)
        if self.add_noise:
            self.obs_buf += (2 * torch.rand_like(self.obs_buf) - 1) * self.noise_scale_vec
    def _reward_contact(self):
        res = torch.zeros(self.num_envs, dtype=torch.float, device=self.device)
        for i in range(self.feet_num):
            is_stance = self.leg_phase[:, i] < 0.55
            contact = self.contact_forces[:, self.feet_indices[i], 2] > 1
            res += ~(contact ^ is_stance)
        return res
    def _reward_feet_swing_height(self):
        contact = torch.norm(self.contact_forces[:, self.feet_indices, :3], dim=2) > 1.
        pos_error = torch.square(self.feet_pos[:, :, 2] - 0.08) * ~contact
        return torch.sum(pos_error, dim=(1))
    def _reward_alive(self):
        return 1.0
    def _reward_contact_no_vel(self):
        contact = torch.norm(self.contact_forces[:, self.feet_indices, :3], dim=2) > 1.
        contact_feet_vel = self.feet_vel * contact.unsqueeze(-1)
        penalize = torch.square(contact_feet_vel[:, :, :3])
        return torch.sum(penalize, dim=(1,2))
    def _reward_hip_pos(self):
```

```
        return torch.sum(torch.square(self.dof_pos[:,[1,2,7,8]]), dim=1)
from legged_gym.envs.base.legged_robot_config import LeggedRobotCfg, LeggedRobotCfgPPO
class tinymalRoughCfg( LeggedRobotCfg ):
    class env:
        num_envs = 2048
        num_observations = 45
        num_privileged_obs = None
        num_actions = 12
        env_spacing = 3.
        send_timeouts = True
        episode_length_s = 20
        test = False
    class init_state( LeggedRobotCfg.init_state ):
        pos = [0.0, 0.0, 0.15]
        default_joint_angles = {
            "LF_HAA": -0.2,
            "LH_HAA": 0.2,
            "RF_HAA": 0.2,
            "RH_HAA": -0.2,
            "LF_HFE": 0.9,
            "LH_HFE": -0.9,
            "RF_HFE": 0.9,
            "RH_HFE": -0.9,
            "LF_KFE": -1.55,
            "LH_KFE": 1.55,
            "RF_KFE": -1.55,
            "RH_KFE": 1.55,
        }
    class commands(LeggedRobotCfg.commands):
        class ranges:
            lin_vel_x = [-0.8, 0.8]
            lin_vel_y = [-0.8, 0.8]
            ang_vel_yaw = [-1.2, 1.2]
            heading = [-3.14, 3.14]
    class control( LeggedRobotCfg.control ):
        control_type = 'P'
        stiffness = {'HAA': 3.3, 'HFE': 3.3, 'KFE': 3.3}
        damping = {'HAA': 0.25, 'HFE': 0.25, 'KFE': 0.25}
        action_scale = 0.25
        decimation = 4
    class asset( LeggedRobotCfg.asset ):
        file = '{LEGGED_GYM_ROOT_DIR}/resources/robots/tinymal/urdf/tinymal.urdf'
        name = "tinymal"
        foot_name = "FOOT"
        penalize_contacts_on = ["thigh", "calf"]
        terminate_after_contacts_on = ["trunk","calf"]
        self_collisions = 0
        flip_visual_attachments = False
    class domain_rand:
        randomize_friction = True
```

```
        friction_range = [0.5, 1.25]
        randomize_base_mass = False
        added_mass_range = [-0.2, 0.5]
        push_robots = True
        push_interval_s = 15
        max_push_vel_xy = 1.
    class rewards( LeggedRobotCfg.rewards ):
        soft_dof_pos_limit = 0.9
        base_height_target = 0.18
        touch_thr= 5
        command_dead = 0.1
        class scales( LeggedRobotCfg.rewards.scales ):
            termination = -0.0
            tracking_lin_vel = 1.0
            tracking_ang_vel = 0.4
            base_height = -0.2
            hip_pos = -0.5
            feet_contact_number=-0.01
            orientation_eular = 0.10
            feet_contact_forces = -0.01
            foot_slip = -0.05
            vel_mismatch_exp = 0.35
            no_fly  = 0.05
class tinymalRoughCfgPPO( LeggedRobotCfgPPO ):
    class algorithm( LeggedRobotCfgPPO.algorithm ):
        entropy_coef = 0.01
    class runner( LeggedRobotCfgPPO.runner ):
        run_name = ''
        max_iterations = 16000
        save_interval = 500
        experiment_name = 'tinymal'
        resume = False
        resume_path =
    "/home/lemon/isaacgym/unitree_rl_gym/logs/rough_go2/May12_22-07-37_/model_1500.pt"
from legged_gym.envs.base.legged_robot import LeggedRobot
from isaacgym.torch_utils import *
from isaacgym import gymtorch, gymapi, gymutil
import torch
class TINYMALRobot(LeggedRobot):
    def compute_observations(self):
        """ Computes observations
        """
        self.obs_buf = torch.cat((
                                    self.base_ang_vel  * self.obs_scales.ang_vel,
                                    self.projected_gravity,
                                    self.commands[:, :3] * self.commands_scale,
                                    (self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos,
                                    self.dof_vel * self.obs_scales.dof_vel,
                                    self.actions
                                    ),dim=-1)
```

```
        if self.add_noise:
            self.obs_buf += (2 * torch.rand_like(self.obs_buf) - 1) * self.noise_scale_vec
    def _init_buffers(self):
        super()._init_buffers()
        rigid_body_state_tensor = self.gym.acquire_rigid_body_state_tensor(self.sim)
        self.rigid_body_states = gymtorch.wrap_tensor(rigid_body_state_tensor).view(self.num_envs,
    -1, 13)
    def _get_noise_scale_vec(self, cfg):
        """ Sets a vector used to scale the noise added to the observations.
            [NOTE]: Must be adapted when changing the observations structure
        Args:
            cfg (Dict): Environment config file
        Returns:
            [torch.Tensor]: Vector of scales used to multiply a uniform distribution in [-1, 1]
        """
        noise_vec = torch.zeros_like(self.obs_buf[0])
        self.add_noise = self.cfg.noise.add_noise
        noise_scales = self.cfg.noise.noise_scales
        noise_level = self.cfg.noise.noise_level
        noise_vec[:3] = noise_scales.ang_vel * noise_level * self.obs_scales.ang_vel
        noise_vec[3:6] = noise_scales.gravity * noise_level
        noise_vec[6:9] = 0.
        noise_vec[9:9+self.num_actions] = noise_scales.dof_pos * noise_level *
    self.obs_scales.dof_pos
        noise_vec[9+self.num_actions:9+2*self.num_actions] = noise_scales.dof_vel * noise_level *
    self.obs_scales.dof_vel
        noise_vec[9+2*self.num_actions:9+3*self.num_actions] = 0.
        return noise_vec
    def _reward_feet_contact_forces(self):
        return torch.sum((torch.norm(self.contact_forces[:, self.feet_indices, :], dim=-1) -
    self.cfg.rewards.max_contact_force).clip(min=0.), dim=1)
    def _reward_hip_pos(self):
        return (0.8-torch.abs(self.commands[:,1])) *  torch.sum(torch.square(self.dof_pos[:, [0, 3,
    6, 9]] - self.default_dof_pos[:, [0, 3, 6, 9]]), dim=1)
    def _reward_feet_contact_number(self):
        """
        Calculates a reward based on the number of feet contacts aligning with the gait phase.
        Rewards or penalizes depending on whether the foot contact matches the expected gait phase.
        """
        contact = self.contact_forces[:, self.feet_indices, 2] > self.cfg.rewards.touch_thr
        contact_num= torch.sum(contact,dim=1)
        reward = torch.where(contact_num <=2, 0, 1)
        return reward* (torch.norm(self.commands[:, :2], dim=1) > 0.1)
    def _reward_orientation_eular(self):
        """
        Calculates the reward for maintaining a flat base orientation. It penalizes deviation
        from the desired base orientation using the base euler angles and the projected gravity
    vector.
        """
        orientation = torch.exp(-torch.norm(self.projected_gravity[:, :2], dim=1) * 20)
```

```
        return orientation
    def _reward_feet_contact_forces(self):
        """
        Calculates the reward for keeping contact forces within a specified range. Penalizes
        high contact forces on the feet.
        """
        return torch.sum((torch.norm(self.contact_forces[:, self.feet_indices, :], dim=-1) -
    self.cfg.rewards.max_contact_force).clip(0, 100), dim=1)
    def _reward_foot_slip(self):
        """
        Calculates the reward for minimizing foot slip. The reward is based on the contact forces
        and the speed of the feet. A contact threshold is used to determine if the foot is in
    contact
        with the ground. The speed of the foot is calculated and scaled by the contact condition.
        """
        contact = self.contact_forces[:, self.feet_indices, 2] > self.cfg.rewards.touch_thr
        foot_speed_norm = torch.norm(self.rigid_body_states[:, self.feet_indices, 10:12], dim=2)
        rew = torch.sqrt(foot_speed_norm)
        rew *= contact
        return torch.sum(rew, dim=1)
    def _reward_vel_mismatch_exp(self):
        """
        Computes a reward based on the mismatch in the robot's linear and angular velocities.
        Encourages the robot to maintain a stable velocity by penalizing large deviations.
        """
        lin_mismatch = torch.exp(-torch.square(self.base_lin_vel[:, 2]) * 10)
        ang_mismatch = torch.exp(-torch.norm(self.base_ang_vel[:, :2], dim=1) * 5.)
        c_update = (lin_mismatch + ang_mismatch) / 2.
        return c_update
    def _reward_no_fly(self):
        contacts = self.contact_forces[:, self.feet_indices, 2] > self.cfg.rewards.touch_thr
        double_contact = torch.sum(1.*contacts, dim=1)==2
        return double_contact*(torch.norm(self.commands[:, :2], dim=1) >
    self.cfg.rewards.command_dead)
import os
LEGGED_GYM_ROOT_DIR = os.path.dirname(os.path.dirname(os.path.realpath(__file__)))
LEGGED_GYM_ENVS_DIR = os.path.join(LEGGED_GYM_ROOT_DIR, 'legged_gym', 'envs')
from legged_gym import LEGGED_GYM_ROOT_DIR, LEGGED_GYM_ENVS_DIR
from legged_gym.envs.tinymal.tinymal_config import tinymalRoughCfg, tinymalRoughCfgPPO
from legged_gym.envs.tinymal.tinymal_env import TINYMALRobot
from legged_gym.envs.go2.go2_config import GO2RoughCfg, GO2RoughCfgPPO, GO2HighLevelCfg,
    GO2HighLevelCfgPPO
from legged_gym.envs.go2.go2_env import GO2Robot
from legged_gym.envs.h1.h1_config import H1RoughCfg, H1RoughCfgPPO
from legged_gym.envs.h1.h1_env import H1Robot
from legged_gym.envs.h1_2.h1_2_config import H1_2RoughCfg, H1_2RoughCfgPPO
from legged_gym.envs.h1_2.h1_2_env import H1_2Robot
from legged_gym.envs.g1.g1_config import G1RoughCfg, G1RoughCfgPPO
from legged_gym.envs.g1.g1_env import G1Robot
from legged_gym.envs.go2.hierarchical_go2_env import HierarchicalGO2Env
```

```
from legged_gym.utils.task_registry import task_registry
task_registry.register( "go2", GO2Robot, GO2RoughCfg(), GO2RoughCfgPPO())
task_registry.register( "go2_high_level", GO2Robot, GO2HighLevelCfg(), GO2HighLevelCfgPPO())
task_registry.register( "tinymal", TINYMALRobot, tinymalRoughCfg(), tinymalRoughCfgPPO())
task_registry.register( "h1", H1Robot, H1RoughCfg(), H1RoughCfgPPO())
task_registry.register( "h1_2", H1_2Robot, H1_2RoughCfg(), H1_2RoughCfgPPO())
task_registry.register( "g1", G1Robot, G1RoughCfg(), G1RoughCfgPPO())
from legged_gym import LEGGED_GYM_ROOT_DIR, envs
import time
from warnings import WarningMessage
import numpy as np
import os
from isaacgym.torch_utils import *
from isaacgym import gymtorch, gymapi, gymutil
import torch
from torch import Tensor
from typing import Tuple, Dict
from legged_gym import LEGGED_GYM_ROOT_DIR
from legged_gym.envs.base.base_task import BaseTask
from legged_gym.utils.math import wrap_to_pi
from legged_gym.utils.isaacgym_utils import get_euler_xyz as get_euler_xyz_in_tensor
from legged_gym.utils.helpers import class_to_dict
from .legged_robot_config import LeggedRobotCfg
class LeggedRobot(BaseTask):
    def __init__(self, cfg: LeggedRobotCfg, sim_params, physics_engine, sim_device, headless):
        """ Parses the provided config file,
            calls create_sim() (which creates, simulation and environments),
            initilizes pytorch buffers used during training
        Args:
            cfg (Dict): Environment config file
            sim_params (gymapi.SimParams): simulation parameters
            physics_engine (gymapi.SimType): gymapi.SIM_PHYSX (must be PhysX)
            device_type (string): 'cuda' or 'cpu'
            device_id (int): 0, 1, ...
            headless (bool): Run without rendering if True
        """
        self.cfg = cfg
        self.sim_params = sim_params
        self.height_samples = None
        self.debug_viz = False
        self.init_done = False
        self._parse_cfg(self.cfg)
        super().__init__(self.cfg, sim_params, physics_engine, sim_device, headless)
        if not self.headless:
            self.set_camera(self.cfg.viewer.pos, self.cfg.viewer.lookat)
        self._init_buffers()
        self._prepare_reward_function()
        self.init_done = True
    def step(self, actions):
        """ Apply actions, simulate, call self.post_physics_step()
```

```
        Args:
            actions (torch.Tensor): Tensor of shape (num_envs, num_actions_per_env)
        """
        clip_actions = self.cfg.normalization.clip_actions
        self.actions = torch.clip(actions, -clip_actions, clip_actions).to(self.device)
        self.render()
        for _ in range(self.cfg.control.decimation):
            self.torques = self._compute_torques(self.actions).view(self.torques.shape)
            self.gym.set_dof_actuation_force_tensor(self.sim, gymtorch.unwrap_tensor(self.torques))
            self.gym.simulate(self.sim)
            if self.cfg.env.test:
                elapsed_time = self.gym.get_elapsed_time(self.sim)
                sim_time = self.gym.get_sim_time(self.sim)
                if sim_time-elapsed_time>0:
                    time.sleep(sim_time-elapsed_time)
            if self.device == 'cpu':
                self.gym.fetch_results(self.sim, True)
            self.gym.refresh_dof_state_tensor(self.sim)
        self.post_physics_step()
        clip_obs = self.cfg.normalization.clip_observations
        self.obs_buf = torch.clip(self.obs_buf, -clip_obs, clip_obs)
        if self.privileged_obs_buf is not None:
            self.privileged_obs_buf = torch.clip(self.privileged_obs_buf, -clip_obs, clip_obs)
        return self.obs_buf, self.privileged_obs_buf, self.rew_buf, self.reset_buf, self.extras
    def post_physics_step(self):
        """ check terminations, compute observations and rewards
            calls self._post_physics_step_callback() for common computations
            calls self._draw_debug_vis() if needed
        """
        self.gym.refresh_actor_root_state_tensor(self.sim)
        self.gym.refresh_net_contact_force_tensor(self.sim)
        self.episode_length_buf += 1
        self.common_step_counter += 1
        self.base_pos[:] = self.root_states[:, 0:3]
        self.base_quat[:] = self.root_states[:, 3:7]
        self.rpy[:] = get_euler_xyz_in_tensor(self.base_quat[:])
        self.base_lin_vel[:] = quat_rotate_inverse(self.base_quat, self.root_states[:, 7:10])
        self.base_ang_vel[:] = quat_rotate_inverse(self.base_quat, self.root_states[:, 10:13])
        self.projected_gravity[:] = quat_rotate_inverse(self.base_quat, self.gravity_vec)
        self._post_physics_step_callback()
        self.check_termination()
        self.compute_reward()
        env_ids = self.reset_buf.nonzero(as_tuple=False).flatten()
        self.reset_idx(env_ids)
        if self.cfg.domain_rand.push_robots:
            self._push_robots()
        self.compute_observations()
        self.last_actions[:] = self.actions[:]
        self.last_dof_vel[:] = self.dof_vel[:]
        self.last_root_vel[:] = self.root_states[:, 7:13]
```

```
    def check_termination(self):
        """ Check if environments need to be reset
        """
        self.reset_buf = torch.any(torch.norm(self.contact_forces[:,
    self.termination_contact_indices, :], dim=-1) > 1., dim=1)
        self.reset_buf |= torch.logical_or(torch.abs(self.rpy[:,1])>1.0,
    torch.abs(self.rpy[:,0])>0.8)
        self.time_out_buf = self.episode_length_buf > self.max_episode_length
        self.reset_buf |= self.time_out_buf
    def reset_idx(self, env_ids):
        """ Reset some environments.
            Calls self._reset_dofs(env_ids), self._reset_root_states(env_ids), and
    self._resample_commands(env_ids)
            [Optional] calls self._update_terrain_curriculum(env_ids),
    self.update_command_curriculum(env_ids) and
            Logs episode info
            Resets some buffers
        Args:
            env_ids (list[int]): List of environment ids which must be reset
        """
        if len(env_ids) == 0:
            return
        self._reset_dofs(env_ids)
        self._reset_root_states(env_ids)
        self._resample_commands(env_ids)
        self.actions[env_ids] = 0.
        self.last_actions[env_ids] = 0.
        self.last_dof_vel[env_ids] = 0.
        self.feet_air_time[env_ids] = 0.
        self.episode_length_buf[env_ids] = 0
        self.reset_buf[env_ids] = 1
        self.extras["episode"] = {}
        for key in self.episode_sums.keys():
            self.extras["episode"]['rew_' + key] = torch.mean(self.episode_sums[key][env_ids]) /
    self.max_episode_length_s
            self.episode_sums[key][env_ids] = 0.
        if self.cfg.commands.curriculum:
            self.extras["episode"]["max_command_x"] = self.command_ranges["lin_vel_x"][1]
        if self.cfg.env.send_timeouts:
            self.extras["time_outs"] = self.time_out_buf
    def compute_reward(self):
        """ Compute rewards
            Calls each reward function which had a non-zero scale (processed in
    self._prepare_reward_function())
            adds each terms to the episode sums and to the total reward
        """
        self.rew_buf[:] = 0.
        for i in range(len(self.reward_functions)):
            name = self.reward_names[i]
            rew = self.reward_functions[i]() * self.reward_scales[name]
```

```
            self.rew_buf += rew
            self.episode_sums[name] += rew
        if self.cfg.rewards.only_positive_rewards:
            self.rew_buf[:] = torch.clip(self.rew_buf[:], min=0.)
        if "termination" in self.reward_scales:
            rew = self._reward_termination() * self.reward_scales["termination"]
            self.rew_buf += rew
            self.episode_sums["termination"] += rew
    def compute_observations(self):
        """ Computes observations
        """
        self.obs_buf = torch.cat((  self.base_lin_vel * self.obs_scales.lin_vel,
                                    self.base_ang_vel  * self.obs_scales.ang_vel,
                                    self.projected_gravity,
                                    self.commands[:, :3] * self.commands_scale,
                                    (self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos,
                                    self.dof_vel * self.obs_scales.dof_vel,
                                    self.actions
                                    ),dim=-1)
        if self.add_noise:
            self.obs_buf += (2 * torch.rand_like(self.obs_buf) - 1) * self.noise_scale_vec
    def create_sim(self):
        """ Creates simulation, terrain and evironments
        """
        self.up_axis_idx = 2
        self.sim = self.gym.create_sim(self.sim_device_id, self.graphics_device_id,
    self.physics_engine, self.sim_params)
        self._create_ground_plane()
        self._create_envs()
    def set_camera(self, position, lookat):
        """ Set camera position and direction
        """
        cam_pos = gymapi.Vec3(position[0], position[1], position[2])
        cam_target = gymapi.Vec3(lookat[0], lookat[1], lookat[2])
        self.gym.viewer_camera_look_at(self.viewer, None, cam_pos, cam_target)
    def _process_rigid_shape_props(self, props, env_id):
        """ Callback allowing to store/change/randomize the rigid shape properties of each
    environment.
            Called During environment creation.
            Base behavior: randomizes the friction of each environment
        Args:
            props (List[gymapi.RigidShapeProperties]): Properties of each shape of the asset
            env_id (int): Environment id
        Returns:
            [List[gymapi.RigidShapeProperties]]: Modified rigid shape properties
        """
        if self.cfg.domain_rand.randomize_friction:
            if env_id==0:
                friction_range = self.cfg.domain_rand.friction_range
                num_buckets = 64
```

```
                bucket_ids = torch.randint(0, num_buckets, (self.num_envs, 1))
                friction_buckets = torch_rand_float(friction_range[0], friction_range[1],
    (num_buckets,1), device='cpu')
                self.friction_coeffs = friction_buckets[bucket_ids]
            for s in range(len(props)):
                props[s].friction = self.friction_coeffs[env_id]
        return props
    def _process_dof_props(self, props, env_id):
        """ Callback allowing to store/change/randomize the DOF properties of each environment.
            Called During environment creation.
            Base behavior: stores position, velocity and torques limits defined in the URDF
        Args:
            props (numpy.array): Properties of each DOF of the asset
            env_id (int): Environment id
        Returns:
            [numpy.array]: Modified DOF properties
        """
        if env_id==0:
            self.dof_pos_limits = torch.zeros(self.num_dof, 2, dtype=torch.float,
    device=self.device, requires_grad=False)
            self.dof_vel_limits = torch.zeros(self.num_dof, dtype=torch.float, device=self.device,
    requires_grad=False)
            self.torque_limits = torch.zeros(self.num_dof, dtype=torch.float, device=self.device,
    requires_grad=False)
            for i in range(len(props)):
                self.dof_pos_limits[i, 0] = props["lower"][i].item()
                self.dof_pos_limits[i, 1] = props["upper"][i].item()
                self.dof_vel_limits[i] = props["velocity"][i].item()
                self.torque_limits[i] = props["effort"][i].item()
                m = (self.dof_pos_limits[i, 0] + self.dof_pos_limits[i, 1]) / 2
                r = self.dof_pos_limits[i, 1] - self.dof_pos_limits[i, 0]
                self.dof_pos_limits[i, 0] = m - 0.5 * r * self.cfg.rewards.soft_dof_pos_limit
                self.dof_pos_limits[i, 1] = m + 0.5 * r * self.cfg.rewards.soft_dof_pos_limit
        return props
    def _process_rigid_body_props(self, props, env_id):
        if self.cfg.domain_rand.randomize_base_mass:
            rng = self.cfg.domain_rand.added_mass_range
            props[0].mass += np.random.uniform(rng[0], rng[1])
        return props
    def _post_physics_step_callback(self):
        """ Callback called before computing terminations, rewards, and observations
            Default behaviour: Compute ang vel command based on target and heading, compute
    measured terrain heights and randomly push robots
        """
        env_ids = (self.episode_length_buf % int(self.cfg.commands.resampling_time /
    self.dt)==0).nonzero(as_tuple=False).flatten()
        self._resample_commands(env_ids)
        if self.cfg.commands.heading_command:
            forward = quat_apply(self.base_quat, self.forward_vec)
            heading = torch.atan2(forward[:, 1], forward[:, 0])
```

```
            self.commands[:, 2] = torch.clip(0.5*wrap_to_pi(self.commands[:, 3] - heading), -1., 1.)
    def _resample_commands(self, env_ids):
        """ Randommly select commands of some environments
        Args:
            env_ids (List[int]): Environments ids for which new commands are needed
        """
        self.commands[env_ids, 0] = torch_rand_float(self.command_ranges["lin_vel_x"][0],
    self.command_ranges["lin_vel_x"][1], (len(env_ids), 1), device=self.device).squeeze(1)
        self.commands[env_ids, 1] = torch_rand_float(self.command_ranges["lin_vel_y"][0],
    self.command_ranges["lin_vel_y"][1], (len(env_ids), 1), device=self.device).squeeze(1)
        if self.cfg.commands.heading_command:
            self.commands[env_ids, 3] = torch_rand_float(self.command_ranges["heading"][0],
    self.command_ranges["heading"][1], (len(env_ids), 1), device=self.device).squeeze(1)
        else:
            self.commands[env_ids, 2] = torch_rand_float(self.command_ranges["ang_vel_yaw"][0],
    self.command_ranges["ang_vel_yaw"][1], (len(env_ids), 1), device=self.device).squeeze(1)
        self.commands[env_ids, :2] *= (torch.norm(self.commands[env_ids, :2], dim=1) >
    0.2).unsqueeze(1)
    def _compute_torques(self, actions):
        """ Compute torques from actions.
            Actions can be interpreted as position or velocity targets given to a PD controller, or
    directly as scaled torques.
            [NOTE]: torques must have the same dimension as the number of DOFs, even if some DOFs
    are not actuated.
        Args:
            actions (torch.Tensor): Actions
        Returns:
            [torch.Tensor]: Torques sent to the simulation
        """
        actions_scaled = actions * self.cfg.control.action_scale
        control_type = self.cfg.control.control_type
        if control_type=="P":
            torques = self.p_gains*(actions_scaled + self.default_dof_pos - self.dof_pos) -
    self.d_gains*self.dof_vel
        elif control_type=="V":
            torques = self.p_gains*(actions_scaled - self.dof_vel) - self.d_gains*(self.dof_vel -
    self.last_dof_vel)/self.sim_params.dt
        elif control_type=="T":
            torques = actions_scaled
        else:
            raise NameError(f"Unknown controller type: {control_type}")
        return torch.clip(torques, -self.torque_limits, self.torque_limits)
    def _reset_dofs(self, env_ids):
        """ Resets DOF position and velocities of selected environmments
        Positions are randomly selected within 0.5:1.5 x default positions.
        Velocities are set to zero.
        Args:
            env_ids (List[int]): Environemnt ids
        """
        self.dof_pos[env_ids] = self.default_dof_pos * torch_rand_float(0.5, 1.5, (len(env_ids),
```

```
    self.num_dof), device=self.device)
        self.dof_vel[env_ids] = 0.
        env_ids_int32 = env_ids.to(dtype=torch.int32)
        self.gym.set_dof_state_tensor_indexed(self.sim,
                                              gymtorch.unwrap_tensor(self.dof_state),
                                              gymtorch.unwrap_tensor(env_ids_int32),
    len(env_ids_int32))
    def _reset_root_states(self, env_ids):
        """ Resets ROOT states position and velocities of selected environmments
            Sets base position based on the curriculum
            Selects randomized base velocities within -0.5:0.5 [m/s, rad/s]
        Args:
            env_ids (List[int]): Environemnt ids
        """
        if self.custom_origins:
            self.root_states[env_ids] = self.base_init_state
            self.root_states[env_ids, :3] += self.env_origins[env_ids]
            self.root_states[env_ids, :2] += torch_rand_float(-1., 1., (len(env_ids), 2),
    device=self.device)
        else:
            self.root_states[env_ids] = self.base_init_state
            self.root_states[env_ids, :3] += self.env_origins[env_ids]
        self.root_states[env_ids, 7:13] = torch_rand_float(-0.5, 0.5, (len(env_ids), 6),
    device=self.device)
        env_ids_int32 = env_ids.to(dtype=torch.int32)
        self.gym.set_actor_root_state_tensor_indexed(self.sim,
                                                     gymtorch.unwrap_tensor(self.root_states),
                                                     gymtorch.unwrap_tensor(env_ids_int32),
    len(env_ids_int32))
    def _push_robots(self):
        """ Random pushes the robots. Emulates an impulse by setting a randomized base velocity.
        """
        env_ids = torch.arange(self.num_envs, device=self.device)
        push_env_ids = env_ids[self.episode_length_buf[env_ids] %
    int(self.cfg.domain_rand.push_interval) == 0]
        if len(push_env_ids) == 0:
            return
        max_vel = self.cfg.domain_rand.max_push_vel_xy
        self.root_states[:, 7:9] = torch_rand_float(-max_vel, max_vel, (self.num_envs, 2),
    device=self.device)
        env_ids_int32 = push_env_ids.to(dtype=torch.int32)
        self.gym.set_actor_root_state_tensor_indexed(self.sim,
                                                     gymtorch.unwrap_tensor(self.root_states),
                                                     gymtorch.unwrap_tensor(env_ids_int32),
    len(env_ids_int32))
    def update_command_curriculum(self, env_ids):
        """ Implements a curriculum of increasing commands
        Args:
            env_ids (List[int]): ids of environments being reset
        """
```

```
        if torch.mean(self.episode_sums["tracking_lin_vel"][env_ids]) / self.max_episode_length >
    0.8 * self.reward_scales["tracking_lin_vel"]:
            self.command_ranges["lin_vel_x"][0] = np.clip(self.command_ranges["lin_vel_x"][0] -
    0.5, -self.cfg.commands.max_curriculum, 0.)
            self.command_ranges["lin_vel_x"][1] = np.clip(self.command_ranges["lin_vel_x"][1] +
    0.5, 0., self.cfg.commands.max_curriculum)
    def _get_noise_scale_vec(self, cfg):
        """ Sets a vector used to scale the noise added to the observations.
            [NOTE]: Must be adapted when changing the observations structure
        Args:
            cfg (Dict): Environment config file
        Returns:
            [torch.Tensor]: Vector of scales used to multiply a uniform distribution in [-1, 1]
        """
        noise_vec = torch.zeros_like(self.obs_buf[0])
        self.add_noise = self.cfg.noise.add_noise
        noise_scales = self.cfg.noise.noise_scales
        noise_level = self.cfg.noise.noise_level
        noise_vec[:3] = noise_scales.lin_vel * noise_level * self.obs_scales.lin_vel
        noise_vec[3:6] = noise_scales.ang_vel * noise_level * self.obs_scales.ang_vel
        noise_vec[6:9] = noise_scales.gravity * noise_level
        noise_vec[9:12] = 0.
        noise_vec[12:12+self.num_actions] = noise_scales.dof_pos * noise_level *
    self.obs_scales.dof_pos
        noise_vec[12+self.num_actions:12+2*self.num_actions] = noise_scales.dof_vel * noise_level *
    self.obs_scales.dof_vel
        noise_vec[12+2*self.num_actions:12+3*self.num_actions] = 0.
        return noise_vec
    def _init_buffers(self):
        """ Initialize torch tensors which will contain simulation states and processed quantities
        """
        actor_root_state = self.gym.acquire_actor_root_state_tensor(self.sim)
        dof_state_tensor = self.gym.acquire_dof_state_tensor(self.sim)
        net_contact_forces = self.gym.acquire_net_contact_force_tensor(self.sim)
        self.gym.refresh_dof_state_tensor(self.sim)
        self.gym.refresh_actor_root_state_tensor(self.sim)
        self.gym.refresh_net_contact_force_tensor(self.sim)
        self.gym.refresh_rigid_body_state_tensor(self.sim)
        self.root_states = gymtorch.wrap_tensor(actor_root_state)
        self.dof_state = gymtorch.wrap_tensor(dof_state_tensor)
        self.dof_pos = self.dof_state.view(self.num_envs, self.num_dof, 2)[..., 0]
        self.dof_vel = self.dof_state.view(self.num_envs, self.num_dof, 2)[..., 1]
        self.base_quat = self.root_states[:, 3:7]
        self.rpy = get_euler_xyz_in_tensor(self.base_quat)
        self.base_pos = self.root_states[:self.num_envs, 0:3]
        self.contact_forces = gymtorch.wrap_tensor(net_contact_forces).view(self.num_envs, -1, 3)
        self.common_step_counter = 0
        self.extras = {}
        self.noise_scale_vec = self._get_noise_scale_vec(self.cfg)
        self.gravity_vec = to_torch(get_axis_params(-1., self.up_axis_idx),
```

```
    device=self.device).repeat((self.num_envs, 1))
        self.forward_vec = to_torch([1., 0., 0.], device=self.device).repeat((self.num_envs, 1))
        self.torques = torch.zeros(self.num_envs, self.num_actions, dtype=torch.float,
    device=self.device, requires_grad=False)
        self.p_gains = torch.zeros(self.num_actions, dtype=torch.float, device=self.device,
    requires_grad=False)
        self.d_gains = torch.zeros(self.num_actions, dtype=torch.float, device=self.device,
    requires_grad=False)
        self.actions = torch.zeros(self.num_envs, self.num_actions, dtype=torch.float,
    device=self.device, requires_grad=False)
        self.last_actions = torch.zeros(self.num_envs, self.num_actions, dtype=torch.float,
    device=self.device, requires_grad=False)
        self.last_dof_vel = torch.zeros_like(self.dof_vel)
        self.last_root_vel = torch.zeros_like(self.root_states[:, 7:13])
        self.commands = torch.zeros(self.num_envs, self.cfg.commands.num_commands,
    dtype=torch.float, device=self.device, requires_grad=False)
        self.commands_scale = torch.tensor([self.obs_scales.lin_vel, self.obs_scales.lin_vel,
    self.obs_scales.ang_vel], device=self.device, requires_grad=False,)
        self.feet_air_time = torch.zeros(self.num_envs, self.feet_indices.shape[0],
    dtype=torch.float, device=self.device, requires_grad=False)
        self.last_contacts = torch.zeros(self.num_envs, len(self.feet_indices), dtype=torch.bool,
    device=self.device, requires_grad=False)
        self.base_lin_vel = quat_rotate_inverse(self.base_quat, self.root_states[:, 7:10])
        self.base_ang_vel = quat_rotate_inverse(self.base_quat, self.root_states[:, 10:13])
        self.projected_gravity = quat_rotate_inverse(self.base_quat, self.gravity_vec)
        self.default_dof_pos = torch.zeros(self.num_dof, dtype=torch.float, device=self.device,
    requires_grad=False)
        for i in range(self.num_dofs):
            name = self.dof_names[i]
            angle = self.cfg.init_state.default_joint_angles[name]
            self.default_dof_pos[i] = angle
            found = False
            for dof_name in self.cfg.control.stiffness.keys():
                if dof_name in name:
                    self.p_gains[i] = self.cfg.control.stiffness[dof_name]
                    self.d_gains[i] = self.cfg.control.damping[dof_name]
                    found = True
            if not found:
                self.p_gains[i] = 0.
                self.d_gains[i] = 0.
                if self.cfg.control.control_type in ["P", "V"]:
                    print(f"PD gain of joint {name} were not defined, setting them to zero")
        self.default_dof_pos = self.default_dof_pos.unsqueeze(0)
    def _prepare_reward_function(self):
        """ Prepares a list of reward functions, whcih will be called to compute the total reward.
            Looks for self._reward_<REWARD_NAME>, where <REWARD_NAME> are names of all non zero
    reward scales in the cfg.
        """
        for key in list(self.reward_scales.keys()):
            scale = self.reward_scales[key]
```

```
            if scale==0:
                self.reward_scales.pop(key)
            else:
                self.reward_scales[key] *= self.dt
        self.reward_functions = []
        self.reward_names = []
        for name, scale in self.reward_scales.items():
            if name=="termination":
                continue
            self.reward_names.append(name)
            name = '_reward_' + name
            self.reward_functions.append(getattr(self, name))
        self.episode_sums = {name: torch.zeros(self.num_envs, dtype=torch.float,
    device=self.device, requires_grad=False)
                             for name in self.reward_scales.keys()}
    def _create_ground_plane(self):
        """ Adds a ground plane to the simulation, sets friction and restitution based on the cfg.
        """
        plane_params = gymapi.PlaneParams()
        plane_params.normal = gymapi.Vec3(0.0, 0.0, 1.0)
        plane_params.static_friction = self.cfg.terrain.static_friction
        plane_params.dynamic_friction = self.cfg.terrain.dynamic_friction
        plane_params.restitution = self.cfg.terrain.restitution
        self.gym.add_ground(self.sim, plane_params)
    def _create_envs(self):
        """ Creates environments:
             1. loads the robot URDF/MJCF asset,
             2. For each environment
                2.1 creates the environment,
                2.2 calls DOF and Rigid shape properties callbacks,
                2.3 create actor with these properties and add them to the env
             3. Store indices of different bodies of the robot
        """
        asset_path = self.cfg.asset.file.format(LEGGED_GYM_ROOT_DIR=LEGGED_GYM_ROOT_DIR)
        asset_root = os.path.dirname(asset_path)
        asset_file = os.path.basename(asset_path)
        asset_options = gymapi.AssetOptions()
        asset_options.default_dof_drive_mode = self.cfg.asset.default_dof_drive_mode
        asset_options.collapse_fixed_joints = self.cfg.asset.collapse_fixed_joints
        asset_options.replace_cylinder_with_capsule = self.cfg.asset.replace_cylinder_with_capsule
        asset_options.flip_visual_attachments = self.cfg.asset.flip_visual_attachments
        asset_options.fix_base_link = self.cfg.asset.fix_base_link
        asset_options.density = self.cfg.asset.density
        asset_options.angular_damping = self.cfg.asset.angular_damping
        asset_options.linear_damping = self.cfg.asset.linear_damping
        asset_options.max_angular_velocity = self.cfg.asset.max_angular_velocity
        asset_options.max_linear_velocity = self.cfg.asset.max_linear_velocity
        asset_options.armature = self.cfg.asset.armature
        asset_options.thickness = self.cfg.asset.thickness
        asset_options.disable_gravity = self.cfg.asset.disable_gravity
```

```
        robot_asset = self.gym.load_asset(self.sim, asset_root, asset_file, asset_options)
        self.num_dof = self.gym.get_asset_dof_count(robot_asset)
        self.num_bodies = self.gym.get_asset_rigid_body_count(robot_asset)
        dof_props_asset = self.gym.get_asset_dof_properties(robot_asset)
        rigid_shape_props_asset = self.gym.get_asset_rigid_shape_properties(robot_asset)
        body_names = self.gym.get_asset_rigid_body_names(robot_asset)
        self.dof_names = self.gym.get_asset_dof_names(robot_asset)
        self.num_bodies = len(body_names)
        self.num_dofs = len(self.dof_names)
        feet_names = [s for s in body_names if self.cfg.asset.foot_name in s]
        penalized_contact_names = []
        for name in self.cfg.asset.penalize_contacts_on:
            penalized_contact_names.extend([s for s in body_names if name in s])
        termination_contact_names = []
        for name in self.cfg.asset.terminate_after_contacts_on:
            termination_contact_names.extend([s for s in body_names if name in s])
        base_init_state_list = self.cfg.init_state.pos + self.cfg.init_state.rot +
    self.cfg.init_state.lin_vel + self.cfg.init_state.ang_vel
        self.base_init_state = to_torch(base_init_state_list, device=self.device,
    requires_grad=False)
        start_pose = gymapi.Transform()
        start_pose.p = gymapi.Vec3(*self.base_init_state[:3])
        self._get_env_origins()
        env_lower = gymapi.Vec3(0., 0., 0.)
        env_upper = gymapi.Vec3(0., 0., 0.)
        self.actor_handles = []
        self.envs = []
        for i in range(self.num_envs):
            env_handle = self.gym.create_env(self.sim, env_lower, env_upper,
    int(np.sqrt(self.num_envs)))
            pos = self.env_origins[i].clone()
            pos[:2] += torch_rand_float(-1., 1., (2,1), device=self.device).squeeze(1)
            start_pose.p = gymapi.Vec3(*pos)
            rigid_shape_props = self._process_rigid_shape_props(rigid_shape_props_asset, i)
            self.gym.set_asset_rigid_shape_properties(robot_asset, rigid_shape_props)
            actor_handle = self.gym.create_actor(env_handle, robot_asset, start_pose,
    self.cfg.asset.name, i, self.cfg.asset.self_collisions, 0)
            dof_props = self._process_dof_props(dof_props_asset, i)
            self.gym.set_actor_dof_properties(env_handle, actor_handle, dof_props)
            body_props = self.gym.get_actor_rigid_body_properties(env_handle, actor_handle)
            body_props = self._process_rigid_body_props(body_props, i)
            self.gym.set_actor_rigid_body_properties(env_handle, actor_handle, body_props,
    recomputeInertia=True)
            self.envs.append(env_handle)
            self.actor_handles.append(actor_handle)
        self.feet_indices = torch.zeros(len(feet_names), dtype=torch.long, device=self.device,
    requires_grad=False)
        for i in range(len(feet_names)):
            self.feet_indices[i] = self.gym.find_actor_rigid_body_handle(self.envs[0],
    self.actor_handles[0], feet_names[i])
```

```
        self.penalised_contact_indices = torch.zeros(len(penalized_contact_names),
    dtype=torch.long, device=self.device, requires_grad=False)
        for i in range(len(penalized_contact_names)):
            self.penalised_contact_indices[i] = self.gym.find_actor_rigid_body_handle(self.envs[0],
    self.actor_handles[0], penalized_contact_names[i])
        self.termination_contact_indices = torch.zeros(len(termination_contact_names),
    dtype=torch.long, device=self.device, requires_grad=False)
        for i in range(len(termination_contact_names)):
            self.termination_contact_indices[i] =
    self.gym.find_actor_rigid_body_handle(self.envs[0], self.actor_handles[0],
    termination_contact_names[i])
    def _get_env_origins(self):
        """ Sets environment origins. On rough terrain the origins are defined by the terrain
    platforms.
            Otherwise create a grid.
        """
        self.custom_origins = False
        self.env_origins = torch.zeros(self.num_envs, 3, device=self.device, requires_grad=False)
        num_cols = np.floor(np.sqrt(self.num_envs))
        num_rows = np.ceil(self.num_envs / num_cols)
        xx, yy = torch.meshgrid(torch.arange(num_rows), torch.arange(num_cols),indexing="xy")
        spacing = self.cfg.env.env_spacing
        self.env_origins[:, 0] = spacing * xx.flatten()[:self.num_envs]
        self.env_origins[:, 1] = spacing * yy.flatten()[:self.num_envs]
        self.env_origins[:, 2] = 0.
    def _parse_cfg(self, cfg):
        self.dt = self.cfg.control.decimation * self.sim_params.dt
        self.obs_scales = self.cfg.normalization.obs_scales
        self.reward_scales = class_to_dict(self.cfg.rewards.scales)
        self.command_ranges = class_to_dict(self.cfg.commands.ranges)
        self.max_episode_length_s = self.cfg.env.episode_length_s
        self.max_episode_length = np.ceil(self.max_episode_length_s / self.dt)
        self.cfg.domain_rand.push_interval = np.ceil(self.cfg.domain_rand.push_interval_s / self.dt)
    def _reward_lin_vel_z(self):
        return torch.square(self.base_lin_vel[:, 2])
    def _reward_ang_vel_xy(self):
        return torch.sum(torch.square(self.base_ang_vel[:, :2]), dim=1)
    def _reward_orientation(self):
        return torch.sum(torch.square(self.projected_gravity[:, :2]), dim=1)
    def _reward_base_height(self):
        base_height = self.root_states[:, 2]
        return torch.square(base_height - self.cfg.rewards.base_height_target)
    def _reward_torques(self):
        return torch.sum(torch.square(self.torques), dim=1)
    def _reward_dof_vel(self):
        return torch.sum(torch.square(self.dof_vel), dim=1)
    def _reward_dof_acc(self):
        return torch.sum(torch.square((self.last_dof_vel - self.dof_vel) / self.dt), dim=1)
    def _reward_action_rate(self):
        return torch.sum(torch.square(self.last_actions - self.actions), dim=1)
```

```
    def _reward_collision(self):
        return torch.sum(1.*(torch.norm(self.contact_forces[:, self.penalised_contact_indices, :],
    dim=-1) > 0.1), dim=1)
    def _reward_termination(self):
        return self.reset_buf * ~self.time_out_buf
    def _reward_dof_pos_limits(self):
        out_of_limits = -(self.dof_pos - self.dof_pos_limits[:, 0]).clip(max=0.)
        out_of_limits += (self.dof_pos - self.dof_pos_limits[:, 1]).clip(min=0.)
        return torch.sum(out_of_limits, dim=1)
    def _reward_dof_vel_limits(self):
        return torch.sum((torch.abs(self.dof_vel) -
    self.dof_vel_limits*self.cfg.rewards.soft_dof_vel_limit).clip(min=0., max=1.), dim=1)
    def _reward_torque_limits(self):
        return torch.sum((torch.abs(self.torques) -
    self.torque_limits*self.cfg.rewards.soft_torque_limit).clip(min=0.), dim=1)
    def _reward_tracking_lin_vel(self):
        lin_vel_error = torch.sum(torch.square(self.commands[:, :2] - self.base_lin_vel[:, :2]),
    dim=1)
        return torch.exp(-lin_vel_error/self.cfg.rewards.tracking_sigma)
    def _reward_tracking_ang_vel(self):
        ang_vel_error = torch.square(self.commands[:, 2] - self.base_ang_vel[:, 2])
        return torch.exp(-ang_vel_error/self.cfg.rewards.tracking_sigma)
    def _reward_feet_air_time(self):
        contact = self.contact_forces[:, self.feet_indices, 2] > 1.
        contact_filt = torch.logical_or(contact, self.last_contacts)
        self.last_contacts = contact
        first_contact = (self.feet_air_time > 0.) * contact_filt
        self.feet_air_time += self.dt
        rew_airTime = torch.sum((self.feet_air_time - 0.5) * first_contact, dim=1)
        rew_airTime *= torch.norm(self.commands[:, :2], dim=1) > 0.1
        self.feet_air_time *= ~contact_filt
        return rew_airTime
    def _reward_stumble(self):
        return torch.any(torch.norm(self.contact_forces[:, self.feet_indices, :2], dim=2) >\
             5 *torch.abs(self.contact_forces[:, self.feet_indices, 2]), dim=1)
    def _reward_stand_still(self):
        return torch.sum(torch.abs(self.dof_pos - self.default_dof_pos), dim=1) *
    (torch.norm(self.commands[:, :2], dim=1) < 0.1)
from .helpers import class_to_dict, get_load_path, get_args, export_policy_as_jit, set_seed,
    update_class_from_dict
from .task_registry import task_registry
from .logger import Logger
from .math import *
from .terrain import Terrain
from .ppo import PPO
from .actor_critic import ActorCritic
from .actor_critic_recurrent import ActorCriticRecurrent
from .on_policy_runner import OnPolicyRunner
from .rollout_storage import RolloutStorage
from .utils import split_and_pad_trajectories, unpad_trajectories
```